

Verifying Provenance of Digital Media: Security Analysis of C2PA and its Implementation

Enis Golaszewski
Cyber Defense Lab
Univ. of Maryland, Balt. Cnty. (UMBC)
Baltimore, Maryland, USA
golaszewski@umbc.edu

Neal Krawetz
Hacker Factor
Fort Collins, Colorado, USA
krawetz26@hackerfactor.com

Alan T. Sherman
Cyber Defense Lab
Univ. of Maryland, Balt. Cnty. (UMBC)
Baltimore, Maryland, USA
sherman@umbc.edu

Edward Ziegler
National Security Agency (NSA)
National Security Agency
Fort George G. Meade, Maryland, USA
evziegl@uwe.nsa.gov

Sai K. Matukumalli
and Roberto Yus
and Carson L. Kegley
Cyber Defense Lab
Univ. of Maryland, Balt. Cnty. (UMBC)
Baltimore, Maryland, USA
{smatuku1, ryus, ckegley1}@umbc.edu

Michael Barthel
and William Bowman
and Bharg Barot
and Kaur Kullman
Cyber Defense Lab
Univ. of Maryland, Balt. Cnty. (UMBC)
Baltimore, Maryland, USA
{mbarthe1, wbowman4,
bhargvb1, kak}@umbc.edu

Index Terms—Formal methods, C2PA, Security and Privacy Protection, CPSA, Protocols

Abstract—Generative AI and advanced editing tools enable malicious actors to create high-quality fake images that can propagate fraud, attack reputations, and manipulate elections. We analyze security properties of the *Coalition for Content Provenance and Authenticity (C2PA)* digital provenance system, which binds cryptographic assertions of provenance to a digital asset, helping users judge its origin.

We analyze three C2PA components: specifications (Version 2.2), selected claim validator implementations, and conformance program (Version 0.1). We state C2PA’s security goals (tamper-evidence of claims and weak file integrity), identify essential additional goals (timestamp agreement, validator consistency, and strong file integrity), review major policies, examine composition with RFC 3161 trusted timestamps, and carry out the first formal-methods analysis of the core protocols.

We show that the specifications and conforming implementations fail to achieve both claimed and essential security goals. Our formal-methods analysis reveals that generators and validators fail to agree on the signature’s trusted timestamp, enabling timestamp alterations that cast doubt on provenance. Inadequate certificate revocation policies cause conforming validators to accept manifests signed with known-compromised certificates, violating *all* security goals. Conforming validators produce inconsistent results; the exclusion range enables undetectable alterations; and the conformance program certifies products without technical review or defined requirements.

Our results show that C2PA does not yet provide the guarantees required for reliable deployment. We suggest concrete improvements, some of which were adopted in the Pixel 10 Pro

and C2PA Version 2.3.

1. Introduction

The ubiquity of AI has rapidly lowered the cost of producing persuasive, high-quality synthetic media [1], [2]. A single actor can fabricate images that appear to depict real events, people, and documents, at a scale and fidelity that strains human judgment and existing forensic tooling. The downstream harms are well understood: fraud enabled by counterfeit evidence, reputation attacks, and political manipulation through fabricated content [3], [4]. In response, some organizations now incorporate *provenance* signals—machine-verifiable metadata intended to help audiences and platforms determine where, when, and how an asset was created or edited. The *Coalition for Content Provenance and Authenticity (C2PA)* system [5] is rapidly emerging as the dominant tool for digital media provenance, yet its actual security guarantees remain poorly understood. We show that the C2PA specifications and ecosystem fail to achieve their stated or essential security goals.

C2PA’s approach is not a single cryptographic primitive but an ecosystem-scale system involving generators, validators, and a conformance program, with protocols for binding assertions to a digital asset, signing and timestamping, certificate and revocation management, and user-facing presentations (Section 2). This complexity introduces a wide surface for exploitation (Sections 5–6): security properties that seem intuitive at the marketing level (e.g., “this photo was taken by a Pixel 10 at this GPS location” or “this timestamp proves that this photo was taken by this date”) can fail given attacker capabilities, incomplete implementations,

ambiguous validation guidance, or diverging trust policies (Section 3). C2PA’s approach also introduces privacy concerns (Section H).

Despite the misleading presence of the word “authenticity” in its name and exaggerated suggestions in its promotional materials (e.g., [6]), C2PA is a mechanism for provenance, not authenticity. *Provenance* refers to the origin and modifications of a digital asset (e.g., *what* device created or modified it, *where*, and *when*). *Authenticity* refers to difficult-to-define properties (what the C2PA specifications refer to as “value judgments”) of the asset, such as “Is it real versus fake”, “Is it true?”, or “Does it faithfully represent something that happened in the real world?” (Section 10).¹ The C2PA specifications admonish users not to infer any value judgments from C2PA claims associated with a digital asset. Instead, the specifications promise only that C2PA validators can *detect tampering of assertions* using a cryptographically signed claim and a *weak notion of file integrity*.

C2PA has substantial institutional momentum and is being adopted in production [7], [8]; C2PA is the approach platforms, journalists, and end users are most likely to encounter today. Real-world provenance is an ecosystem problem that depends on more than cryptographic soundness: specifications and engineering details, interoperability, trust and revocation management, validator policy, and user-facing presentation all matter. Accordingly, we analyze C2PA as implemented and validated in the wild, emphasizing the gap between conformance and effective assurance.

Our analysis focuses on the security of C2PA 2.2 across *design, implementation, and use*.² We evaluate the C2PA specifications, including their normative requirements, validation guidelines, trust management, privacy considerations, and RFC 3161 timestamp integration. We analyze the specifications focusing on *content credentials (Cr)* (Section 4), but exclude soft-bindings or auxiliary technologies, which are currently unimplemented. We complement this specification-level analysis with empirical validator experiments. We also examine a deployed implementation: the Google Pixel 10 Pro C2PA pipeline, which C2PA currently lists as a conforming product at the maximum assurance level. Our approach is not merely to identify isolated bugs, but to determine whether C2PA—as deployed—provides the consistency and evidentiary value that users and platforms require.

To our knowledge, our work is the first comprehensive security analysis of C2PA and the first formal-methods analysis of its underlying cryptographic protocols. Our analysis is challenging in part because the C2PA specifications are expansive, complex, and evolving: Version 2.2 has 248 pages, with over 20 associated documents totaling over 2,500 pages, not including source code or related standards (e.g., JUMBF, CBOR, JPEG, PNG, XMP, and EXIF).

1. The C2PA specifications use the noun “authenticity” in a nonstandard narrow way to mean the verification of a digital signature (authentication).

2. C2PA releases an updated specification roughly twice a year. We focus on version 2.2 because, at the time of this writing, there are no conforming products or complete implementations for C2PA’s versions 2.3 and 2.4.

Experience shows that there is great value in formal-methods analysis of cryptographic protocols, including in the design phases, because humans are poorly suited to analyze protocols’ complex nuanced security properties and their many possible execution sequences.

Assuming valid public-key certificates, our formal-methods symbolic analysis proves strong agreement on a claim signature, but not on the signature’s trusted timestamp. In practice, however, neither this tamper-evident property of claims—nor *any* of the security goals—holds because conforming validators accept revoked, malicious certificates (Figure 4). Failure to agree on a claim signature’s trusted timestamp enables adversaries to modify or replace timestamps without detection, undermining provenance. We discovered this timestamp vulnerability through our formal-methods analysis and demonstrate it in Figure 2.

As Kocher [9] explains, when any security system that uses public-key infrastructure accepts revoked signing certificates, the system risks permitting an adversary to defeat all security goals. Violating best practices, the C2PA specifications permit validators to skip checking for revoked signing certificates (e.g., certificates corresponding to compromised private-keys). In a prudent effort to balance privacy and security, the specifications intentionally make this egregious design choice. The specifications [5, Section 15.9] permit validators optionally to perform certificate revocation checks, but only with the *Online Certificate Status Protocol (OCSP)*, which compromises privacy (Section H). While safeguarding privacy is an important consideration, skipping revocation checks is unacceptable, and using *certificate revocation lists (CRLs)* would yield a better privacy-security tradeoff. Figure 4 illustrates a serious consequence of this major vulnerability of the C2PA specifications.

Our contributions include: (1) Evaluating the C2PA specifications holistically, including requirements, validation guidance, trust management, privacy considerations, and integration with RFC 3161 timestamping. (2) Formalizing C2PA security goals (tamper-evidence and weak file integrity) and identifying essential additions: timestamp agreement, validator consistency, and strong file integrity. (3) Performing the first formal-methods analysis of C2PA protocols using the *Cryptographic Protocol Shapes Analyzer (CPSA)* [10]. (4) Presenting examples of digital assets that violate one or more of the C2PA security goals stated in the paper. For each asset’s Cr, we analyze the assertions and evaluate the claim by using multiple public validators. (5) Assessing the conformance program and the inconsistency between conforming validators: Adobe’s *Content Authenticity Inspect* and Sanmarcsoft’s *Verifieddit*. (6) Proposing concrete specifications and ecosystem changes based on our findings. (7) We explain how to achieve more robust file integrity (Sections 8, 13).

The Pixel 10 Pro and C2PA Version 2.3 (January 2026) and Version 2.4 (April 2026) incorporated some of our recommendations.

All of our artifacts are available on GitHub [11], including our CPSA input models and output shapes. Several appendices provide additional detailed information.

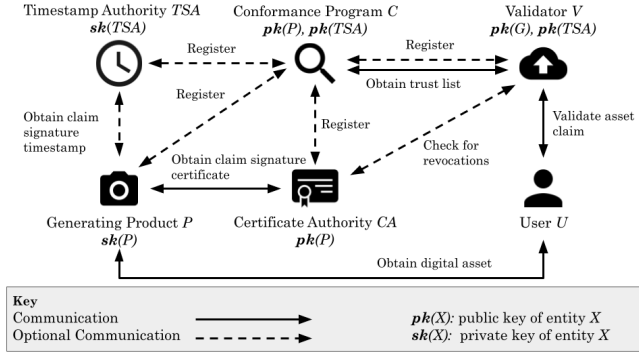


Figure 1. C2PA ecosystem involving users, generators, validators, timestamp authorities, and certificate authorities. Components may register with the conformance program.

2. Overview of the C2PA Ecosystem

The *Coalition for Content Provenance and Authenticity* (C2PA) [5] addresses misinformation by specifying the framework and management of digital asset provenance. Founded in 2021, the C2PA initiative was originally led by Adobe and Microsoft, with additional steering committee members including Arm, Intel, Truepic, and the BBC [12]. In 2024, the steering committee expanded to include Google [13], Meta [14], and Amazon [15]. It now reports more than 300 general members, spanning device manufacturers, media organizations, and security-focused companies. Interest is not limited to industry: in January 2025, the U.S. National Security Agency, the Australian Signals Directorate’s Australian Cyber Security Centre, the Canadian Centre for Cyber Security, and the U.K. National Cyber Security Centre jointly released guidance on content credentials and emphasized the importance of widespread adoption [16]. Despite widespread interest in this problem space, many participating organizations, including some on the steering committee, have not yet deployed C2PA-enabled solutions or have only limited pilot implementations.

The C2PA organization provides the specifications and related services, including the authoritative certificate list, official list of protocol references (*bindings*), and *conforming products list*. C2PA works closely with the Adobe-provided *Content Authenticity Initiative* (CAI) [17], which functions as the outreach branch for C2PA and provides working implementations, including *c2patool* [18] and the *Content Authenticity Verify* online validator [19].

Components. The C2PA ecosystem involves content generators, validators, users, a trusted timestamp authority (TSA), a certificate authority (CA), and a product conformance program (Figure 1).

The C2PA specifications employ a system called *Content Credentials* (Cr), which combines X.509-based cryptographic signatures with structured provenance information (referred to as *claims* or *assertions*). C2PA’s security considerations [20] define a trust model in which the user assumes that the signed assertions were generated by the signer. This

trust model makes no judgment concerning the veracity of the assertions. Types of permissible assertions have evolved across versions of the C2PA specifications. Early versions (February 2021 to May 2025) allowed user-supplied and unverified data to be commingled with verified information.

Assertions. We analyze Version 2.2 of the specifications (May 2025) [5]. C2PA 2.2 distinguishes between *created* and *gathered* assertions. *Created assertions* originate from trusted sources, through vetted hardware or software. *Gathered assertions* derive from unverified sources, such as user-provided names or organizational information. Because trusted hardware capable of generating reliable metadata are not yet widely available, most attributes, including image contents, lens settings, GPS coordinates, and timestamps, are currently recorded as gathered assertions.

Exclusions. Each assertion is cryptographically hashed and signed using an X.509 signature. This process provides identity attribution, cryptographic validation, and tamper detection for the signed data. In addition, the asset file and manifest are hashed and signed, but this protection omits the “*exclusion range*” (set of bit positions) of these data. The specifications are remarkably permissive in what may be excluded: “*the claim generator shall ensure the exclusion range only contains content from C2PA Manifest Store, or asset metadata (e.g., EXIF, IPTC metadata).*” Consequently, an adversary can modify bits in the asset’s metadata or tamper with the asset’s manifest store without invalidating the claim signature. For this reason, C2PA’s claimed security goal of file integrity is very weak (Section 5).

The specifications omit claim manifests from the hash of an asset’s metadata, enabling an adversary to tamper with the order and contents of the asset’s manifest store. This omission is necessary to resolve a circular dependency: a claim manifest resides within the asset, and the asset’s hash value resides within the claim manifest. We uncovered two potential issues resulting from this policy. First, an adversary who modifies the manifest store, such as reordering or removing claim manifests, violates the asset’s overall file integrity (Section 5). Second, changes to the ordering of claim manifests result in errors and crashes some validating software (Section 13.1).

C2PA’s rationale [21] for allowing an exclusion range is based on privacy: it would like to allow a user to redact private information (e.g., GPS location) from an asset’s metadata after asset generation. C2PA’s intent is to allow content anonymization after signing and without requiring resigning—a feature that prioritizes editorial flexibility over strict data integrity. In Sections 8 and 13, we propose a more robust way involving additional signatures to navigate this tension between provenance and privacy.

Auxiliary technologies. The C2PA organization recommends pairing Cr with auxiliary technologies, such as watermarking and fingerprinting. These auxiliary approaches lie outside the scope of this paper, which focuses on the Cr mechanism.

Verifying content credentials. Validator tools help users analyze manifests of digital assets by examining the Cr. Given the complexity and relative newness of C2PA, there are few publicly-available validators. The most complete is *c2patool* [18] by the CAI. This command-line tool validates signatures and supports inserting and extracting Cr metadata. Due to current rapid release cycles [18], different versions of the tool may generate different validation results for the same C2PA-signed media. As of March 2026, the following web-based validators include:

- *Content Authenticity Inspect* [22] by Adobe offers the most up-to-date of the available web-based tools. The current version displays only minimal certificate and created assertion information. Certificate chains and some unverified assertions are not reported. Moreover, it does not support older versions of the C2PA specifications—images signed six months ago may not be verifiable.
- *Verify* [19] by the CAI [23] is similar to Adobe’s *Inspect*, but appears to be a few revisions older and not updated as frequently.
- *ProofCheck* [24] by ProofMode determines whether media contains a Cr record, but performs very little verification. It readily displays user-supplied untrusted information as verified.
- *Verifieddit* [25] by Sanmarcsoft, a CAI member, displays more details than any other validator, including raw output similar to *c2patool*.

There are and have been other web-based C2PA validators, but these are often left unmaintained after an initial period of enthusiastic support, while others have already been taken offline, including Truepic’s *Content Credentials* and Microsoft’s *Content Integrity* services. Among the web-based validators that we list, *Content Authenticity Inspect* and *Verifieddit* are on the conforming products list. In addition, the conforming products list [26] includes other validators that are not publicly accessible.

Between the rapidly evolving specifications, lack of regular maintenance, and updating for validation options, there are significant inconsistencies between the available validation tools. A single signed picture may generate contradictory results depending on the tool used and when it was used (Figure 3).

Conformance program. The C2PA conformance program provides a list of conforming products. These signature generators, devices, and validators have been reviewed by the conformance program and found to comply with the specifications. There are currently two assurance levels: Level 1 focuses on the correct implementation of the C2PA protocol and typically relies on software-based protections for the signing keys. Level 2 focuses on hardware-backed security solutions. Achieving an assurance level does not mean that the implementation is correct or complete: the current C2PA review process evaluates only product descriptions, not software or hardware implementations.

Provided as a JSON-encoded file [27], the conforming product list includes specification versions and assurance levels. We found no evidence, however, that any of the

available products check the conformance list. Hence, non-technical end-users may find it difficult to determine if a valid signature is from a conforming product or if the validator is a conforming product. Moreover, users cannot easily discern the specification version in their media, nor if a validator is equipped to process assets from conflicting versions.

The Google Pixel 10 family of smartphones were the first devices to receive Assurance Level 2 [8]. This designation means that the keys are protected by hardware-backed security, and an end-user can reliably determine that the media was signed by a Pixel 10. However, the Pixel 10’s implementation includes many limitations—a strong signature around untrusted data does not make the data more trustworthy.

In December 2025, the conforming products list [27] included five validators, including Adobe *Inspect*. As of March 2026, CAI’s *c2patool* and *Verify* were not listed.

3. Examples of Security Failures

We present three representative examples that illustrate how the C2PA specifications and ecosystem fail to achieve any of security goals SG1–SG5 stated in Section 5. Each example features a digital asset with an associated Cr. Example 1—a generative AI output with a substituted timestamp—shows that validators cannot identify trusted timestamp alterations, violating SG3 (*timestamp agreement*). This failure results from the specifications not binding a claim to its signed timestamp. Example 2—a generative AI output (from ChatGPT-4o) using an earlier C2PA version—demonstrates that validators can produce inconsistent results, violating SG4 (*validator consistency*). Example 3—a well-known camera forgery produced by generative AI—illustrates that validators can accept altered images when they are signed by a revoked certificate. Because conforming C2PA validators can accept revoked certificates, C2PA does not satisfy any of the five security goals: using revoked or compromised certificates, adversaries can modify claims and any bits of asset files without detection. For additional examples, see Appendix F.

For each example, we identify compromised security goals, analyze what information can be extracted from the Cr claims, and explain how public online C2PA validators interpret and present the Cr to end users. We also discuss potential security hazards resulting from these failures.

To verify a Cr, we followed the four-step validator workflow: (1) determine the Cr’s C2PA specification version, (2) ensure that the Cr complies with the corresponding C2PA specifications, (3) verify the claim’s *hash.data* assertion matches the image data, and (4) authenticate each claim’s signature and (if present) the timestamp.

Table 1 summarizes the contradictory results we obtained from conforming validators by CAI [19] and *Verifieddit* [25], and from validators by Adobe [22] and *ProofMode* [24], which (as of March 24, 2026) were not listed on the conforming products list. We performed all computations using the validator’s web interfaces.

TABLE 1. VERIFICATION RESULTS FOR THREE IMAGE-MANIFEST EXAMPLES THAT ILLUSTRATE HOW THE C2PA SYSTEM FAILS TO ACHIEVE ANY OF THE SECURITY GOALS. THE RIGHTMOST FOUR COLUMNS SHOW RESULTS FROM CONFORMING VALIDATORS BY ADOBE AND VERIFIEDDIT, AND FROM VALIDATORS BY CAI AND PROOFMODE, WHICH ARE NOT ON THE CONFORMING PRODUCTS LIST. IN EXAMPLE 1, NONE OF THE VALIDATORS RECOGNIZE THAT THE TIMESTAMP WAS SWAPPED, VIOLATING SECURITY GOAL SG3. IN EXAMPLE 2, VALIDATORS PRODUCE INCONSISTENT RESULTS ON THIS VALID IMAGE, VIOLATING SG4. IN EXAMPLE 3, THREE OF THE VALIDATORS FAIL TO DETECT THAT THE FORGED IMAGE WAS SIGNED BY A REVOKED SIGNATURE CERTIFICATE, VIOLATING SG1–SG5.

Ex.	Description	Version	Media Age	Adobe	Verifieddit	CAI	ProofMode
1	GenAI with Swapped Timestamp	2.2	1 month	Valid, AI	Valid, Untrusted, AI	Valid, AI	Valid, Not AI
2	ChatGPT-4o, No Timestamp	2.x	7 months	Not Found	Expired	Not Found	Valid, Not AI
3	Nikon Z6 III Camera Forgery	2.x	6 months	Valid, Not AI	Invalid, Untrusted	Valid, Not AI	Valid, Not AI

3.1. Example 1: Timestamp Replacement

Example 1 (Figure 2) shows that conforming validators fail to detect replaced timestamps, violating SG3. The image is a JPEG photo of a vehicle that was generated using Microsoft’s Azure OpenAI ImageGen. The example includes a Cr with a new timestamp replacing the claim signature’s original timestamp. To replace the timestamp we developed a custom tool [28]. This example also illustrates that conforming validators do not identify when images are created by generators that are not on the conforming products list.

The Cr in this example comprises a single signed claim with a corresponding signature timestamp. The claim contains two assertions: (1) a hard-binding *c2pa.hash.bboxes* assertion to the underlying asset, and (2) a created *c2pa.actions.v2* assertion that Azure OpenAI ImageGen created the image on February 23, 2026. The modified claim signature states that the Microsoft Corporation signed the claim on March 20, 2026. The original signature, however, is from February 23; the March date is from our substituted timestamp that used the DigiCert TSA (a conforming CA).

As of March 24, 2026, Azure OpenAI ImageGen does not appear on C2PA’s public conformance lists. The four validators indicate, however, that the photo’s Cr is valid, including the replaced timestamp. Verifieddit indicates that the Cr is valid, but indicates that the signer (Microsoft) is untrusted on both the original and altered files. We theorize that Verifieddit restricts itself to certificates in the public conformance trust list. CAI, Adobe, and ProofMode continue to acknowledge entries on the *interim trust list (ITL)*, which the CAI froze on January 1, 2026. ProofMode is the only tested validator that marked the picture as not being

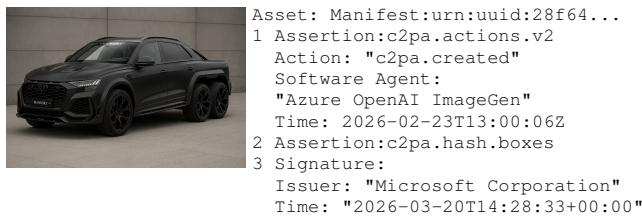


Figure 2. A C2PA-enabled photo generated by Microsoft’s Azure OpenAI ImageGen. We replaced the photo’s original claim signature timestamp (applied by Microsoft) with a replacement timestamp which we obtained from a legitimate TSA (DigiCert). Violating SG3, none of the validators tested identified the replaced timestamp or the discrepancy between the image creation date (February 23) and the signature date (March 20).

AI generated. As a result, validators disagree on whether to trust the signing certificate and whether it is AI. Each of the validators incorrectly attribute the trusted timestamp to Microsoft, instead of DigiCert.

Example 1 illustrates four issues. (1) It demonstrates that an adversary can replace a timestamp on a claim signature without detection. Consequently, the adversary can cast doubt on the time at which a claim generator signed a claim. Aided by the “optional” revocation checking (see Example 3), an adversary that compromises a trusted TSA (or pushes a malicious TSA through the conformance program) may replace a legitimate timestamp with a fraudulent, back-dated timestamp, enabling them to discredit a legitimate Cr. (2) Validators—including conforming validators—return inconsistent results for the same media. (3) The conformance program does not specify which trust lists validators must use, and permits alternate trust lists, including deprecated trust lists containing revoked, expired, or malicious certificates (Section 3.3). (4) Validators do not identify which trust lists they use.

3.2. Example 2: Inconsistent Validators

Example 2 (Figure 3) illustrates that conforming validators can produce contradictory results, violating SG4. OpenAI’s GPT-4o generated this image of a lantern. The image is not malicious but confuses validators for three reasons: (1) The image uses an older version of C2PA that predates Version 2.2 and the conformance program. (2) The image incorporates a Cr with nested claims that lack claim signature timestamps. (3) The claim signatures correspond to an expired (Truepic) root certificate.

The image’s Cr comprises two claims: an outer *active* claim that provides provenance for the image’s thumbnail, and an inner claim for the full-sized image. Neither claim signature incorporates a signed timestamp. The active claim includes three assertions: (1) *c2pa.thumbnail.ingredient.jpeg*, which incorporates binary data for a thumbnail corresponding to the image, (2) *c2pa.hash.data*, a hard-binding to the underlying asset, and (3) *c2pa.ingredient.v3*, an assertion that the thumbnail corresponds to the inner claim’s image. The inner claim includes a created *c2pa.actions.v2* assertion indicating two actions. First, GPT-4o generated the full-sized image. Second, the OpenAI API applied some unspecified conversion to the image. Finally, the outer claim includes a hard-binding *c2pa.hash.data* assertion to the image data. OpenAI signed



```
Asset: Manifest:urn:c2pa:02c9d2ff...
1 Assertion:c2pa.thumbnail...
2 Assertion:c2pa.ingredient.v3
  Title: "image.png"
  Relationship: "componentOf"
3 Assertion:c2pa.hash.data
4 Manifest:urn:c2pa:93190d09-...
  Title: "image.png"
  4a Assertion:c2pa.actions.v2
    Action1: "c2pa.created",
    SoftwareAgent: "GPT-4o"
    Action2: "c2pa.converted",
    SoftwareAgent: "OpenAI API"
  4b Assertion:c2pa.hash.data
  4c Signature:
    Issuer: "OpenAI", Time: null
5 Signature:
  Issuer: "OpenAI", Time: null
```

Figure 3. A lantern graphic generated by ChatGPT-4o together with a corresponding simplified C2PA claim manifest. The validators we tested returned contradictory results, violating SG4. Assertion 1 defines a thumbnail image for the claim, which validators may display to the user. Assertion 2 indicates that the asset includes an ingredient, for which there is an ingredient claim manifest. The ingredient manifest 4 indicates that GPT-4o created the original image, and that OpenAI API converted the image to its current format. OpenAI signed the claim and the ingredient claim. Neither claim signature has a timestamp.

both claims, but omitted trusted timestamps. Each claim signature is currently expired and invalid.

Attempting to validate this Cr produces contradictory results: ProofMode states that the Cr is valid; CAI and Adobe fail to detect the Cr; and Verifieddit states correctly that the signing certificates are expired. ProofMode and Verifieddit fail to indicate that OpenAI is not on the conforming products list. This example highlights a problematic future for aged Crs: users will be unable to verify them due to expired certificates, outdated Cr formats, and missing timestamps.

3.3. Example 3: Revoked Certificates

Example 3 (Figure 4) carries a valid signature from a revoked Nikon certificate; accepting it allows the adversary to modify claims and file contents without detection. Exploiting a flaw in the Nikon Z6 III camera’s multiple exposure feature, this forgery demonstrates how an adversary can violate all C2PA security goals SG1–SG5.

The flaw, discovered by Adam Horshack [29], enables an adversary to produce an arbitrary image with a corresponding valid Cr carrying the camera’s signature. Following Horshack’s disclosure, Nikon revoked its signing certificate on September 24, 2025. Despite Nikon’s revocation, three out of four validators in our experiment continue to validate Horshack’s forgery six months later on March 25, 2026. Accepting a revoked signing certificate enables an adversary to modify the claims and file contents without detection. This example also produces inconsistent validator results.

Example 3’s Cr includes appropriate assertions, including an identity assertion attributing the forgery to its creator (Adam Horshack), a *c2pa.actions* assertion indicating Horshack’s camera created the image, a *c2pa.hash.bboxes* hard-binding, and a valid signature from Nikon. The claim



```
Asset: Manifest:urn:uuid:a3bc6f...
1 Assertion:stds.schema-org.Cre...
  Type: "Person"
  Author: "Horshack"
2 Assertion:c2pa.actions
  Action: "c2pa.created"
  Name: "Horshack"
3 Assertion:c2pa.thumbnail...
4 Assertion:c2pa.hash.bboxes
5 Signature:
  Issuer: "NIKON CORPORATION"
  Time: null
```

Figure 4. A forged generative AI image originating from a compromised Nikon Z6 III camera. The image’s Cr carries a valid signature backed by a revoked certificate, which our tested validators fail to detect. Accepting a revoked certificate allows an adversary to violate all security goals S1–S5. This Cr includes an identity assertion *stds.schema-org.CreativeWork*, which attributes the forgery to its creator Adam Horshack. Horshack created this forgery by exploiting a flaw in Nikon Z6 III’s multiple exposure feature, which enables an adversary to superimpose an arbitrary, malicious image as an overlay for a legitimate image. Subsequently, the camera applies a valid Cr to this malicious composition.

signature does not include a trusted timestamp. By accepting the revoked Nikon certificate, validators verify the signature.

The CAI, Adobe, and ProofMode validators state the Cr for Horshack’s forgery is valid. Verifieddit indicates that the Cr is invalid, likely because Verifieddit does not reference entries on the ITL. Because the specifications do not strictly require a full check for revocation, none of the validators indicate that Nikon’s C2PA certificate has been revoked. Accepting a revoked certificate constitutes a serious security vulnerability.

On January 1, 2026, C2PA deprecated the ITL, noting “No new entries will be added, and no updates will be made.” [30]. Despite this pronouncement, CAI issued three subsequent updates, most recently in March 2026 [31]. Although the latest revision appears to remove the Nikon certificate, validators—including Adobe’s conforming validator—continue to accept it. Unannounced changes to a supposedly frozen trust list ensures that different validations will produce different results based on when they cached the list, violating SG4.

4. C2PA Specifications

The C2PA specifications (Version 2.2) define a framework in which *content generators* create digital assets (e.g., images), Crs that incorporate cryptographic hashes, and digital signatures to bind provenance data to digital assets. C2PA’s approach assumes that a user will use *validators* to leverage a digital asset’s Cr, in combination with traditional approaches such as fact-checking, to verify the asset’s provenance. The specifications focus on cryptographic security goals (integrity, authentication, non-repudiation) of Crs and defers the choice of provenance data within a Cr to the content generator’s C2PA implementation.

Ecosystem. The specifications define a complex ecosystem that includes users, content generators, and validators (Figure 1). These entities produce validation flows, Crs, claims, assertions, and claim signatures, and optionally depend on

a trusted *timestamp authority* (TSA), the *conformance program*, and a *certificate authority* (CA). Content generators create digital assets and populate the Crs related to the media. The generators include physical cameras, software packages, and generative AI systems. The signing certificates may be provided by device manufacturers, news organizations, journalists, and other entities that output digital media. *Validators* help users by verifying Crs of digital assets.

Asset generation and validation flows. As shown in Figure 5, content generators create digital assets. The *application programming interface* (API) populates the Cr and attaches it to the asset. Given an asset and a Cr, a validator verifies the Cr’s cryptographic signatures.

Users validate each assertion to determine whether an adversary tampered with the assertion, and that the assertion complies with C2PA’s rules. Rogue content generators may create assertions that comply with the specification, validate correctly, but present dishonest, misleading, incomplete, or unhelpful information. For such assertions, the user might make a *value judgment* based on the available information (e.g., other assertions in the assertion store and contextual information outside the Cr). The value judgment determines the rightness or wrongness of the information signed by the cryptographic signature. Sections 10 and G discuss potential risks and limitations facing users when making these judgments.

Content credentials. A Cr contains one or more *claims*. A content generator embeds the Cr in the media’s metadata (alongside other standard metadata structures, such as *Exchangeable Image File Format* (EXIF), XMP, and IPTC) or refers to a remote URL containing a Cr (referred to as a *sidecar*).

Claims. A typical claim of a Cr includes the following

information:

- 1) The claim identifier (*instance ID*).
- 2) Information about the claim generator, such as the generator’s name.
- 3) A URI reference to a valid claim signature object.
- 4) A list of URIs and corresponding URI hash values. These are required for each created assertion and optional for each gathered assertion.
- 5) A list of assertions (*assertion store*).
- 6) The claim signature object.

A claim combines assertions with information about the claim generator, and is cryptographically signed. The *claim manifest* of a digital asset is a collection of claims.

Assertions. Each Cr incorporates a list of assertions, known as an *assertion store*, which comprise facts that the *claim generator* asserts about the corresponding digital asset. As noted in Section 2, the C2PA specifications define two types of assertions: *created* and *gathered*. Example assertions include an asset’s creation time, a cryptographic hash of the asset, the physical device or software that created the asset, the software packages that edited the asset, and whether the asset’s creation involved generative AI. The specifications require that a standard manifest contain a created assertion specifying that a generator created or modified the digital asset; all other assertions are optional.

To cryptographically bind a Cr to the corresponding digital assets, the C2PA specifications identify a class of assertions called *hard-bindings*. Each Cr’s assertion store must include exactly one hard-binding assertion. Currently, the C2PA specifications describe hard-binding assertions that incorporate cryptographic hashes of digital assets. The most common hard-binding is the *data hash* assertion, which does not specify any media format. According to the specifications, it is suitable only when a more specific hashing assertion for the digital asset is unavailable.

Claim signatures. To prevent an adversary from tampering with the claim, each Cr incorporates the claim generator’s digital signature of the claim’s hash, and optionally, a trusted timestamp of the claim signature (RFC 3161 [32]). The claim signature is used by a verifier to determine what assertions were placed on a given asset.

Timestamps. The claim generator obtains a timestamp by sending the hash of the claim signature to a TSA and requesting a signed timestamp. Claim generators with an internal TSA, or an active connection to an online TSA, can do so immediately. Alternatively, a claim generator may request a timestamp later when it establishes a connection.

The purpose of the timestamp is to provide evidence of a time *by which* the generator signed the claim; the timestamp is not evidence of when the generator created the asset. Validators use timestamps to determine, when the TSA attested to counter-signing the signature, the generator’s claim signing certificate was valid (e.g., not expired or revoked). C2PA Version 2.2 weakly advises the signer to use a RFC 3161 compliant TSA “if possible”.

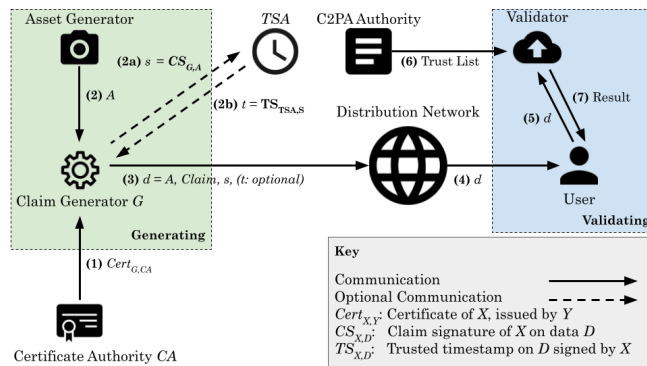


Figure 5. Asset generation and validation flows in the C2PA ecosystem. The asset A receives a claim from the claim generator G , which incorporates an optional timestamp from a trusted timestamping authority TSA . Subsequently, G publishes the asset A together with the signed claim (we call this combination d), to a distribution network (e.g., the Internet). Subsequently, a user downloads d , then verifies the claim contents using a public validator. The public validator references a C2PA trust list (e.g., the conformance trust list) to determine whether to trust G ’s signature of the claim, as well as the optional TSA signature on the claim timestamp.

5. Security Goals

To guide our analysis, we articulate five security goals—two goals specified by C2PA, and three additional essential goals that the C2PA system should also satisfy. This section states and explains these goals. We derive SG1 and SG2 from the C2PA specifications:

- SG1 *Claim integrity*. Given any claim associated with any digital asset, validators are able to verify the integrity and origin of the assertions.
- SG2 *Weak file integrity*. Given any digital asset protected by C2PA and its associated manifest, validators are able to verify the integrity of the data explicitly covered by the cryptographic signature. (This goal deliberately omits all data residing within the specified “exclusion byte ranges,” rendering the overall file integrity verification incomplete and susceptible to undetected modification.)

We propose additional essential goals for cryptographic provenance systems such as C2PA:

- SG3 *Timestamp agreement*. When any claim generator includes a trusted timestamp, the validators are able to verify the timestamp’s integrity, the timestamp’s origin, and the claim generator that associated the timestamp with the claim.
- SG4 *Validator consistency*. Given any claim, distinct validators sharing the same specification version and a trust list agree on the validity the claim.
- SG5 *Strong file integrity*. Given any digital asset protected by C2PA and its claim manifest, validators are able to verify the integrity of the digital asset (including non-C2PA metadata).

We analyze the C2PA system against SG1–SG5, including a formal-methods analysis of its core protocols against SG1, SG3, and SG5.

Goals specified by C2PA. From the C2PA specifications and related C2PA documents, we extract two security goals: SG1 and SG2. The specifications allude to SG1 (*claim integrity*) as the primary security goal of Content Credentials, stating: “*the assertions included within can be validated as associated with the underlying asset, correctly formed, and free from tampering.*” While not stated as an explicit security goal, numerous passages of the specifications discuss integrity of an asset’s bits and included metadata. We formulate this secondary security goal as SG2 (*weak file integrity*). Based on the specifications, C2PA 2.2 promises to achieve only SG1 and SG2. These goals apply *only* to the non-excluded portions of the manifest, which includes a digest of the non-excluded part of the asset file.

The C2PA specifications present their weak file integrity security goal vaguely and inconsistently. For any asset, which the specifications define to include metadata, the specifications state that hard-bindings enable validators to ensure that “*the asset has not been modified, by determining values that can match only this asset and no other, not even other assets derived from it or renditions produced from it.*” [5, Section 9.1]. Contradicting the previous statement,

the specifications then indicate that a claim generator “*may exclude asset metadata (i.e., metadata outside a C2PA Manifest such as EXIF or XMP) from the content binding.*” [5, Section 9.2.5].

Additionally, misleading C2PA promotional materials (e.g., [6]) hint at stronger file integrity properties than C2PA achieves. On March 5, 2026, while answering a question about file integrity at the end of a panel presentation about C2PA to congressional staff, lobbyists, and policymakers, CAI Director and C2PA Steering Committee member Andy Parsons [33] mischaracterized C2PA’s file integrity property by failing to mention the exclusion range (Section 2 and Example 5 in Appendix F).

Although users desire the ability to identify an asset’s authenticity (e.g., Is the image real?), it is not a goal of the C2PA specifications to do so, and it is not possible for C2PA-like systems (Section 10). The specifications explicitly reject such a goal, stating “*C2PA specifications SHOULD NOT provide value judgments about whether a given set of provenance data are ‘good’ or ‘bad.’*” [5, Section 1.2].

Additional essential goals.

We consider SG3, SG4, and SG5 essential for any cryptographic provenance system, and C2PA is remiss in omitting them: Timestamp disagreement (SG3) enables adversaries to backdate claims using a malicious TSA or post-date with any TSA. Validator inconsistency (SG4) prevents conforming validators from agreeing on a manifest’s validity under a common specification version and trust list, confusing users. SG5 expands SG2 to cover the entire asset, including metadata outside the exclusion range. We recommend an approach to modify metadata for privacy while preserving strong file integrity (Section 13.2).

6. Adversarial Model

We assume a malicious model in which protocol communicants do not have to follow the protocol. Specifically, we use the *Dolev-Yao (DY)* network intruder model [34]. In the DY model, the adversary has full control over all messages on the network. The adversary can execute and manipulate an arbitrary number of sessions simultaneously. The adversary can insert, delete, resend, and mix messages and data from separate or simultaneous executions. Importantly, the adversary may pose as a legitimate party on the network. The adversary can compute cryptographic functions for which they know the data and key. The adversary, however, cannot break standard cryptographic primitives.

Adversary objectives include: (A) undetectably altering claim manifest data, including assertions and timestamps; (B) misleading a verifier regarding asset provenance by adding, altering, or removing assertions; (C) adjusting the asset timestamp; or (D) altering data before signing to introduce confusion, such as by adjusting capture time or applying a camera filter.

7. Formal-Methods Analysis (Summary)

Using the *Cryptographic Protocol Shapes Analyzer* (CPSA) [10], we perform a formal-methods analysis of the core C2PA protocols involving three roles: *Generator G*, *Trusted Timestamp Authority TSA*, and *Validator V*, against the security goals SG1, SG2, SG3, and SG5. We do not model SG4 because that would require comparing outputs from multiple validators. We perform our analysis assuming that the public-key certificates are valid, a wishful assumption that does not hold in practice because the specifications do not require validators to check for revoked certificates.

We define and analyze five protocol models. Derived from C2PA specifications, *C2PA-NoTimestamp* and *C2PA-RemoteTSA* model manifests with no timestamp and with remote TSA-provided timestamps, respectively. The remaining three models explore possible alternative protocols: *C2PA-GeneratorSignature* requires the generator to sign the TSA timestamp; *C2PA-TimestampManifest* places the timestamp directly in the manifest; and *C2PA-TimestampManifestF* adds an assertion (“flag”) in the parent manifest to ensure the timestamp manifest exists. We recommend *C2PA-GeneratorSignature* and *C2PA-TimestampManifestF*.

We carry out our analysis in five steps: (1) We formally state security goals SG1, SG2, SG3, and SG5. (2) We model the protocols in the strand space model. (3) For each protocol, we define session contexts. (4) For each protocol role, we specify origination assumptions. (5) Using CPSA, we verify whether the formal security goals hold for all possible executions of the protocols.

CPSA exhaustively searches for all unique protocol executions (“shapes”) within a strand space, verifying if the security goal is satisfied in each. These searches include intruder strands with unbounded sessions, considering all distinct protocol-interaction attacks. For each search, CPSA terminated, thereby achieving provable definite results [35]: it either finds a counterexample or yields a proof by exhaustive search (see Appendix E).

Table 2 summarizes our results. Our analysis reveals that the *C2PA-RemoteTSA* and *C2PA-TimestampManifest* protocols do not satisfy SG3 (timestamp agreement). Due to inadequate bindings, a structural weakness in the *C2PA-RemoteTSA* protocol results in *G* and *V* not agreeing on the timestamp or *TSA*. The *C2PA-TimestampManifest* protocol reaches agreement between *V* and *G* on the timestamp and *TSA*. However, it and the *C2PA-RemoteTSA* protocols fail to satisfy SG3 because the timestamp can be removed with the remaining manifest still validating. By contrast, our suggested improvements satisfy SG1, SG2, SG3, and SG5.

8. Our Two Suggested Improved Protocols

From our formal-methods analysis, we identify two possible approaches for improving the core C2PA protocol to satisfy security goal SG3. Our first approach (*C2PA-GeneratorSignature*) modifies the specified C2PA timestamp protocol to include (1) a signature over the timestamp by

TABLE 2. SUMMARY OF FORMAL-METHODS ANALYSIS OF CORE C2PA PROTOCOLS FOR SG1 (CLAIM INTEGRITY), SG2 (WEAK FILE INTEGRITY), SG3 (TIMESTAMP AGREEMENT), AND SG5 (STRONG FILE INTEGRITY) FROM THE PERSPECTIVE OF THE VALIDATORS *V*, ASSUMING ALL PUBLIC-KEY CERTIFICATES ARE VALID. A CHECKMARK (✓) INDICATES THAT THE SECURITY GOAL IS MET. A CROSSMARK (✗) INDICATES THAT THE SECURITY GOAL DOES NOT HOLD. A CIRCLE (○) INDICATES THAT THE SECURITY GOAL HOLDS IF AND ONLY IF THE ENTIRE ASSET IS PROTECTED. THE INTEGER *i* DENOTES THE MODEL INDEX.

<i>i</i>	Validator	SG1	SG2	SG3	SG5
0	C2PA-NoTimestamp	✓	✓	n/a	○
1	C2PA-RemoteTSA	✓	✓	✗	○
2	C2PA-GeneratorSignature*	✓	✓	✓	✓
3	C2PA-TimestampManifest	✓	✓	✗	○
4	C2PA-TimestampManifestF*	✓	✓	✓	✓

(* Our suggested improvements)

the generator, and (2) an assertion in the generator’s timestamped claim that a timestamp should be present. Simply signing the timestamp is insufficient to satisfy SG3 as the timestamp could be removed and the claim would still be valid. The validator would be unable to detect that the timestamp was removed. By including an assertion that a claim has a timestamp in the claim that was timestamped, a validator would be able to detect removal of the timestamp.

Our second approach (*C2PA-TimestampManifestF*) involves using an update manifest [5, Section 11.2.3] to add a timestamp assertion, rather than using a separate signing function for timestamps. Similar to our first approach, the parent manifest promises (via an assertion) a future update manifest which appends a timestamp assertion. Without this promise, validators may validate the parent manifest (without a timestamp) without detecting that an adversary removed the update manifest with the timestamp assertion.

SG5 (*strong file integrity*) is an important goal to prevent undetectable modifications to digital assets. Therefore, in our improved protocols we modeled and recommend that the entire digital asset receives cryptographic protection.

9. Analysis of the C2PA Conformance Program

The C2PA Conformance Program [27] “provides assurance that products adhere to the Content Credentials specification, and fulfill a set of security requirements to ensure they are producing and validating C2PA data correctly.” While these goals are achievable, the current implementation lacks critical assurances, specifically regarding the *Certificate Authority Trust List* and *Conforming Products List*.

9.1. Certificate Authority Trust List

To manage trust, C2PA specifies *trust lists*, which list the root certificates of trusted CAs. The C2PA conformance program [27] maintains a pair of trust lists specifying trusted CAs and TSAs that passed the conformance program requirements. Each entry in the trust list corresponds to a

conforming product, which achieves a maximum assurance level of 1 or 2. A certificate may exist at a lower assurance level than the maximum assurance level of its corresponding conforming product. C2PA allows for on-device TSAs, such as the on-board TSA within the Pixel 10 Pro. Such a TSA must attempt to synchronize with a trusted time source at least once every 24 hours.

C2PA specifies numerous requirements for issuing leaf certificates, including requiring two trusted individuals (e.g., CA system operator, PKI administrator) to carry out the operation. For example, the Pixel 10 Pro automatically generates leaf certificates that sign each claim. C2PA certificate policy includes requirements for personnel controls (e.g., background checks, training), archival policies, compromise and disaster recovery, key-pair generation and installation, key storage, and network security. It is unclear how C2PA determines if a CA complies with these requirements.

Additionally, CAs must respond appropriately with the OCSP to indicate certificate revocation. C2PA does not maintain, nor require CAs to maintain, CRLs. In practice, validators respond very slowly (if at all) to revocations. For example, when Nikon revoked its certificates following an attack on their C2PA implementation [36], several validators continued to accept them. The conforming Adobe Inspect [22] still reported that a Nikon forgery with a revoked certificate was valid six months later (March 2026).

C2PA trust lists require periodic status refreshes that are enforced via maximum validity periods for signing and TSA certificates. At Assurance Level 1, claim signing leaf certificates are valid for 366 days, while Level 2 reduces this to 90 days. TSA leaf and intermediate certificates have maximum validity periods of 4110 days (11.2 years) and 5479 days (15 years), respectively. Per the conformance program’s TSA trust list [27], the maximum validity period of any TSA root certificate is 30 years, though most expire sooner.

The specifications permit validators to accept a claim signature corresponding to an expired certificate if (1) the claim signature carries a trusted timestamp that falls within the matching certificate’s validity period, and (2) the trusted timestamp “falls within the validity period of the TSA’s signing certificate and all CA certificates up to the trust anchor.” [5, Section 15.8.2]. To our knowledge, C2PA currently does not specify any means by which to “re-certify” expired claims. Therefore, any claim will become impossible to verify after at most 30 years.

9.2. Conforming Products List

C2PA’s *conforming products list* consists of a JSON file posted on their GitHub [27]. The list contains information on the applicant, whether the product is currently conformant, what type of product it is (generator or verifier), the assurance level, and, as of C2PA Version 2.3 based on our feedback, the specification version. The list does not include the date updated, though GitHub versioning provides the date of last commit. The specifications do not require validators to check the conforming products list,

and we found no evidence that validators perform such a check. Furthermore, non-technical users will likely find it challenging to parse this list.

While the Pixel 10 family currently stands as the only smartphone capable of acting as a C2PA Level 2 generator, the hardware itself does not appear on the C2PA conforming products list. Instead, the certification is held by the “Pixel Camera” application (certified 2025-06-27), which postdates the Pixel 9 series.

Responding to our feedback, C2PA’s revised Version 2.3 [37] introduced an optional field in the conforming products JSON entitled `specVersion`, identifying the C2PA specification version with which the product is compatible. This improvement, however, does not solve the problem of different cameras within a camera family.

The current conforming products list does not allow for any versioning adjustments where the product name is insufficiently descriptive, and thus fails to distinguish conforming from non-conforming products.

10. Assessing Content and Provenance: Issues, Limits, and Capabilities

When assessing media, users seek to evaluate digital object *authenticity* and distinguish altered from genuine content. While provenance systems cannot fully guarantee authenticity, they provide essential assurances regarding creation time, location, device, attribution, and edit history. Foundational elements of these systems include cryptography (e.g., encryption, signature, hashing, PKI infrastructure), trusted hardware (e.g., TPMs), time and location services (e.g., TSA, GPS), validator tools, conformance programs, and reputation systems of creators and distributors. However, these systems face inherent tensions, such as between provenance and privacy (Appendix H). Importantly, while cryptography detects tampering, it cannot prove payload truthfulness.

All provenance systems are fundamentally limited by their inability to control capture device inputs. Adversaries may take a *photo of a photo*, photograph staged events, or frame misleadingly incomplete scenes. C2PA understandably ignores this challenge. Even known capture metadata cannot resolve recapture issues, and while detection tools exist, advanced AI and editing tools can further convolute any interpretation. This challenge was recently illustrated by controversial images of Venezuelan leader Nicolás Maduro [38]. Optical illusions add further complexity.

Another difficult issue arises from capture devices using AI-enabled cameras. For example, Google Pixel 10 family and Apple iPhone cameras default to applying AI-powered software that processes information captured through the camera lens. This practice leaves trace signatures identifying AI alterations on source photos, making it difficult to separate reality from fiction in the outputs of capture devices [39], [40].

Provenance systems must clearly articulate what they do and do not provide, and they must deliver sufficient levels of

assurance. It is counterproductive, dangerous, and harmful for such systems to provide misleading signals, which might encourage a user to reach false conclusions (e.g., believing that a fake image is genuine, or disbelieving a genuine image [41]). Also, it creates confusion when validators yield contradictory results.

A user’s ability to discern provenance is limited by their knowledge, skills, and available resources, so validators must be highly usable, reliably convey meaningful information, and be well-maintained. Provenance systems also depend on trustworthy supporting services (timestamps, GPS, certificates, and conforming products lists), each with serious limitations. For example, civilian GPS signals can be spoofed [42] and the C2PA conformance program does not examine source code or implementations.

Dealing with aging digital assets is a separate challenge: specifications might become obsolete; certificates might expire; and cryptographic primitives might be broken. Provenance systems must deal with these challenges.

11. Previous and Related Work

To our knowledge, our study is the first comprehensive security analysis of C2PA, the first formal-methods analysis of C2PA protocols, and the first to evaluate Crs of real photos to determine the system’s benefits and limitations.

The U.S. Department of Defense [16] assessed content credentials, including C2PA in the context of AI-generated misinformation. Bushey et al. [43] placed C2PA provenance features within the broader domain of computational archival science. Earnshaw et al. [44] analyzed C2PA from a broadcast industry perspective. Krawetz [45], [46] demonstrated many limitations of C2PA, including how users can misinterpret the implications of tamper evident. Fisher [47] analyzed the flow and removal of C2PA metadata through distribution channels, and the World Privacy Forum evaluated the privacy implications [48]. Jess Weatherbed [49] discussed multiple limitations in a non-technical review of C2PA.

Petrangeli et al. [50] introduced a client-side implementation of C2PA validation for video streaming. Fotos et al. [51] extended the C2PA framework with new privacy features, and Monica et al. [52] addressed privacy concerns related to C2PA-enabled photo edits using zero-knowledge proofs. Park et al. [53] presented the Chimera attack strategy that aims to fool deepfake detectors and image recapture detectors. Microsoft [54] highlights some of C2PA’s limitations and advocates a multi-signal verification strategy that combines Crs with other imperfect solutions, such as watermarking and perceptual hashing.

Other approaches for addressing media provenance include: (1) labeling media in the metadata, such as IPTC’s Digital Source Type [55] and the XMP standard [56] for tracking provenance and edits, (2) visible and invisible watermarking, which permits identifying if the media are AI generated or altered, such as Google SynthID [57], Adobe TrustMark [58], or Meta Stable Signature [59], among others, (3) cryptographic notary approaches that assess whether

the file existed and has not been altered, such as SEAL [60] and various distributed ledger approaches [61], [62], and (4) personal attestations, such as those defined by *Creator Assertions Working Group (CAWG)* [63] and *W3C Verifiable Credentials (VCs)* [64]. Many of these technologies can be combined for layered provenance and authentication, for example, combining invisible content watermarking with IPTC’s label, XMP’s history, and SEAL’s notarization.

12. Open Problems and Future Work

Amid rising misinformation, digital forgery, and shifting C2PA specifications, many open problems remain in assisting users with digital asset assessment.

(1) Enhance the usability of validators by helping users understand claim assertions and their implications. Enhance the utility of validators by helping users (to the extent possible) measure the veracity of claim assertions. To this end, it would be useful to integrate C2PA with services for reputation, fact-checking, and directing users to the original sources of digital assets.

(2) Design, build, and test more effective C2PA validators that fully comply with the specifications and better support users in assessing provenance.

(3) Analyze additional aspects of C2PA, including security of Version 2.3 soft-bindings (e.g., watermarking), privacy leaks resulting from ingredients and assertions (or redacted assertions), and applications to non-photographic digital media (e.g., video, audio). Perform security reviews of conforming C2PA products, including the EZDRM (DRM as a service), which incorporates C2PA for live video feeds at Assurance-Level 2.

(4) Explore other cryptographic strategies to combat digital misinformation (e.g., digital notary services such as SEAL) and compare these to C2PA.

(5) Investigate how AI can support provenance and authenticity systems notwithstanding that AI sometimes lies and hallucinates.

As future work, we plan to leverage our experience analyzing C2PA to design and build a more capable validator.

13. Findings and Recommendations

Our findings identify violated security goals, reporting history, and mitigation recommendations.

13.1. Findings

1) *Timestamps*. Our formal-methods analysis reveals that a claim signature does not bind to its trusted timestamp. As demonstrated in Section 3.1, an adversary can remove or substitute the timestamp without detection. Additionally, the specifications permit validators, such as CAI’s open-source C2PA validation framework, to validate assets with malformed timestamps, providing only an informational warning and not an error. According to the specifications, these intentional design decisions enable

generators to create C2PA claims and timestamp them later (e.g., when taking a photo in an area with no network access to a TSA). This finding violates SG3. *We reported this issue to C2PA via email on October 27, 2025 and discussed the issue in subsequent meetings with them. This weakness persists in Version 2.3 of the specifications.*

- 2) *Version number.* C2PA manifests do not include a field indicating their version. Consequently, validators cannot work correctly, and they process outdated manifests inconsistently—sometimes dangerously (Figure 3). This finding violates SG4.

We reported this issue to C2PA on September 30, 2025. C2PA 2.3 includes a version field but does not require generators to use it and does not report an error if the version field does not match the Cr feature set.

- 3) *Certificate revocation.* Although C2PA 2.2 suggests checking for revoked certificates, CAI’s open-source validator software [18], [65] does not check for revocations by default. Validators that rely on this package will accept revoked or expired certificates. For example, after a publicized attack on Nikon’s C2PA, Nikon revoked their certificate on September 24, 2025 [36]. Nevertheless, public validators (including conforming validators) continued to deem Nikon-signed claims valid through January 14, 2026 and beyond. Similarly, C2PA 2.3 suggests that validators ascertain and reject revoked certificates. This version, however, supports only OCSP [37], [66] (which undermines privacy) and does not support revocation lists. This finding directly violates SG4 and indirectly all security goals SG1–SG5 as a result of accepting revoked certificates.

We reported this issue to C2PA on September 30, 2025.

- 4) *Metadata.* Motivated in part by protecting privacy, generators are required only to include a creation assertion identifying what generator created or modified the digital asset, and a hard-binding to the asset’s binary data. Many implementations include only these required assertions and do not protect useful metadata (e.g., time, location, EXIF, XMP, and IPTC metadata fields), which can assist users and experts in assessing provenance. For example, the Pixel 10 omits camera information from the manifest, and the initial release omitted EXIF data from the cryptographic protection. This finding complicates a user’s ability to assess provenance of a digital asset.

We reported this issue to Google and C2PA on October 28, 2025. Version 2.3 of the C2PA specifications did not update the required assertions. In December 2025, Google updated the Pixel 10 [67] to include large portions of the EXIF metadata in their hard-bindings, thereby incorporating the data into the manifest.

- 5) *File integrity.* The manifest permits excluding specific byte ranges of the media from cryptographic protection and allows for the redaction of signed assertions. Any data not protected by hashing can be exploited by an adversary to alter information or remove attribution without invalidating the signature.

We reported this issue to C2PA on March 24, 2026.

- 6) *Manifest labels.* The C2PA specifications permit nested manifests with unique labels, and recommend omitting the manifest from the cryptographic signature [5, Section 10.4.1]. This combination creates a blind-spot, where the labels can be altered without detection. By manipulating the label names so that they are not unique, we produced images that crashed validators based on CAI’s open source c2patool. Other validators (e.g., CAI, Adobe) returned errors when processing these manifests. This finding violates SG4 and SG5.

We reported this issue on October 28, 2025, and a bug was submitted on November 7, 2025 [68]. CAI fixed the segmentation fault in the source code, but C2PA 2.3 did not modify their guidance to validators concerning conflicting label processing.

- 7) *Privacy.* The C2PA specifications permit redacting content for privacy without violating the signature. However, data that can be redacted can also be altered to mislead the user.

We discussed this issue with C2PA representatives on August 11, 2025.

13.2. Recommendations

Based on our evaluation of the C2PA 2.2 specifications, we make the following recommendations:

- 1) *Timestamps.* Require the signer to incorporate the timestamp as an assertion within the claim. For the case of generating a claim signature offline (without access to a TSA), require claim generators to generate update manifests, which include a timestamp assertion, when online. Require verifiers to report malformed timestamps as errors.
- 2) *Version number.* Generators and validators must use the version field. Manifest entries that do not comply with the stated version in the conforming products list must be marked as an error.
- 3) *Certificate revocation.* Require validators to determine the revocation status of all certificates (e.g., using CRLs) and reject all revoked certificates. Support CRLs. Require validators to identify to users which trust list(s) they are using. Specify security requirements for conforming validators.
- 4) *Metadata.* Encourage generators to include additional verified assertions, including time, location, EXIF, XMP, and IPTC metadata fields.
- 5) *File integrity.* Following the principle of least privilege, the exclusion range must be restricted to the minimum required by the file format and application context. Excluding data outside of the manifest, or in unprotected manifest portions, enables undetectable alterations and impact provenance.
- 6) *Manifest labels.* As with *file integrity*, the signature should cover the entire manifest, only excluding the signature.
- 7) *Privacy.* Post-capture changes to the media for anonymity or privacy must require resigning with a notation of the edit.

14. Conclusion

Our study uncovers seven serious problems with the C2PA specifications, validator implementations, and conformance program. For example, our formal-methods analysis shows that C2PA claim generators and validators fail to agree on the claim signature’s trusted timestamp. Furthermore, the specifications’ inadequate certificate revocation policies allow validators to accept manifests signed by compromised certificates, which allows a DY adversary to violate *all* five security goals. Due in part to insufficient information in claims (e.g., version), ambiguity in identifying which manifest is active, and inappropriately permissive policies, conforming validator implementations sometimes produce contradictory conclusions. The C2PA conformance program certifies products without defining security requirements for conforming validators and without reviewing source code.

There has been a history of overstating what C2PA is capable of achieving (Appendix I). At best, and *only* assuming that valid certificates are used, the C2PA specifications achieve SG1 (*tamper evidence of claims*) and SG2 (*weak file integrity*). Even with this strong assumption, SG1 and SG2 are only narrowly true: parts of the digital asset file unprotected by digital signature (the exclusion range) can be modified without detection (Section 2). Furthermore, that promise is insufficient to address the need to assess the provenance of digital assets: it is also important to assure timestamp agreement and validator consistency (Section 5). Furthermore, an untampered false, ambiguous, or meaningless claim is useless, and potentially harmful, when backed up by a C2PA verified signature.

While there are fundamental limits to what a system like C2PA can possibly achieve to assist users in evaluating provenance, more can and should be done. There is tension between privacy and provenance, but concern for privacy is no excuse for several of the engineering mistakes (e.g., disallowing privacy-preserving methods for checking for revoked certificates (Section 4)).

We recommend several ways to strengthen the specifications, some of which have been implemented in the Pixel 10 (December 2025 update [67]), C2PA 2.3 [37], and C2PA 2.4 [69]. However, the specification Version 2.4 (April 2026) does not resolve any of our concerns. For example, Version 2.3 added that the generator “may” include a version number, and 2.4 changed it to “should”, but the requirement ought to be “must”. We hope our remaining recommendations will be adopted by later specification versions and future verifiers.

On March 5, 2026, during a panel presentation about C2PA to congressional staff, lobbyists, and policymakers, Adobe’s Global Head of Content Authenticity Andy Parsons overstated, “... *Now this is ready. This technology works. It’s been battle tested. It’s on some phones. It’s in cameras. Um, make sure anybody you meet ... in your travels through Congress is aware that this technology is ready. It works. It’s proven. And now we need to put it to work.*” [33, at time 1:09:05 in video].

Unfortunately, as we have shown, neither the long complex C2PA specifications, C2PA validators, nor C2PA conformance program yet provide the guarantees required for reliable deployment or standards adoption. For example, the specifications inadequately deal with timestamps, certificate revocation, and version numbers. There are insufficient guidelines for validators and conforming validators produce inconsistent results. The conformance program does not check validator functionality, examine source code, or specify validator security requirements. Although the underlying *cryptography* in C2PA is battle tested, its *integration* in the specifications is not and has failed our tests. There is a danger that, in the presence of misinformation, an unreliable tool prematurely rushed for adoption could increase confusion rather than provide clarity. Marketing and commercial aspirations should not taint scientific judgment. More careful cryptographic design, engineering, and analysis is needed.

The C2PA ecosystem rests on the specifications. As we demonstrate, serious vulnerabilities in the specifications undermine the ecosystem. Additionally, there are deficiencies with the implementations and user interfaces. C2PA could eventually be positioned to become a widely adopted useful tool for helping people to assess the provenance of digital assets. We hope that our study and recommendations will accelerate this process and strengthen C2PA.

Acknowledgments

Sherman was supported in part by the National Science Foundation (NSF) under DGE grants 1753681 (SFS) and 2138921 (SaTC). Sherman and Yus were supported by NSF under DGE grant 2438185 (Cyber+AI). Sherman and Golaszewski were supported by the UMBC cybersecurity exploratory grant program. Kaur, Golaszewski, Sherman, and Yus were supported by UMBC’s CIDER Grant Program. We thank Jonathan Fuchs for informative insights and feedback.

References

- [1] J. Goldstein, J. Chao, S. Grossman, A. Stamos, and M. Tomz, “How Persuasive Is AI-Generated Propaganda?,” *Stanford Institute for Human-Centered Artificial Intelligence*, 2024. Accessed: 2026-01-15.
- [2] S. Guo, Y. Zhong, and X. Hu, “People are more susceptible to misinformation with realistic AI-synthesized images that provide strong evidence to headlines,” *Harvard Kennedy School Misinformation Review*, 2025.
- [3] E. L. Green, “‘They couldn’t break me’: A protester, the White House and a doctored photo,” *The New York Times Company*, February 2026.
- [4] AFP, “Trump posts, then deletes, video that includes racist depiction of Obamas as apes,” *The Times of Israel*, February 2026.
- [5] Coalition for Content Provenance and Authenticity, “C2PA technical specification, Version 2.2.” https://spec.c2pa.org/specifications/specifications/2.2/specs/_attachments/C2PA_Specification.pdf. C2PA Specifications. Accessed: 2026-4-12.
- [6] Adobe, “Content credentials.” <https://lion.app.box.com/s/rang889q86q1jb80i3phxiww6mzlwcyj>, 2025. 3 min video promotion.
- [7] ISO, “ISO/DIS 22144, Authenticity of information – Content Credentials.” <https://www.asist.org/2025/03/19/iso-22144-authenticity-information-standards/>. Accessed: 2026-01-15.
- [8] E. Lynch and S. Hanna, “How Pixel and Android are bringing a new level of trust to your images with C2PA Content Credentials.” <https://security.googleblog.com/2025/09/pixel-android-trusted-images-c2pa-content-credentials.html>. Accessed: 2025-12-28.
- [9] P. C. Kocher, “On certificate revocation and validation,” in *International conference on financial cryptography*, pp. 172–177, Springer, 1998.
- [10] M. D. Liskov, J. D. Ramsdell, J. D. Guttman, and P. D. Rowe, *The Cryptographic Protocol Shapes Analyzer: A Manual*, 2016.
- [11] REDACTED, “Anonymous GitHub repository.” <https://anonymous.4open.science/r/c2pa-1FC1/README.md>, January 2026.
- [12] C2PA, “C2PA founding press release.” <https://c2pa.org/c2pa-founding-press-release/>. Accessed: 2025-12-29.
- [13] C2PA, “Google to join C2PA to help increase transparency around digital content.” <https://c2pa.org/google-to-join-c2pa-to-help-increase-transparency-around-digital-content/>. Accessed: 2025-12-29.
- [14] C2PA, “Meta Joins the C2PA Steering Committee.” https://spec.c2pa.org/post/meta_pr/. Accessed: 2025-12-29.
- [15] C2PA, “Amazon Joins the C2PA Steering Committee.” <https://c2pa.org/amazon-joins-the-c2pa-steering-committee/>. Accessed: 2026-01-20.
- [16] US National Security Agency, Australian Signals Directorate’s Australian Cyber Security Centre, Canadian Centre for Cyber Security, and United Kingdom National CyberSecurity Centre, “Content Credentials: Strengthening Multimedia Integrity in the Generative AI Era.” <https://media.defense.gov/2025/Jan/29/2003634788/-1/1/0/CSI-CONTENT-CREDENTIALS.PDF>. Accessed: 2025-12-29.
- [17] Adobe, “Content Authenticity Initiative.” <https://contentauthenticity.org/>. Accessed: 2026-3-24.
- [18] Adobe, “C2PA Rust Library.” <https://github.com/contentauth/c2pa-rs>. Accessed: 2025-12-29.
- [19] Coalition for Content Provenance and Authenticity, “Content Credentials.” <https://verify.contentauthenticity.org>. Accessed: 2026-3-24.
- [20] Coalition for Content Provenance and Authenticity, “C2PA Security Considerations.” https://spec.c2pa.org/specifications/specifications/2.2/security/Security_Considerations.html. Accessed: 2025-12-28.
- [21] C2PA, “C2PA Explainer 1.3.” https://spec.c2pa.org/specifications/specifications/1.3/explainer/_attachments/Explainer.pdf. Accessed: 2026-01-13.
- [22] Adobe, “Adobe Content Authenticity Inspect.” <https://contentauthenticity.adobe.com/inspect>. Accessed: 2026-3-24.
- [23] Adobe, “Introducing Adobe Content Authenticity: A free web app to help creators protect their work, gain attribution and build trust.” <https://blog.adobe.com/en/publish/2024/10/08/introducing-adobe-content-authenticity-free-web-app-help-creators-protect-their-work-gain-attribution-build-trust>. Accessed: 2025-12-29.
- [24] Guardian Project, “ProofCheck: Verification by Proofmode.” <https://check.proofmode.org/>. Accessed: 2026-3-24.
- [25] Sanmarcsoft, “Verifieddit.” <https://www.verifieddit.com/>. Accessed: 2026-3-24.
- [26] Coalition for Content Provenance and Authenticity, “Conformance Explorer.” <https://spec.c2pa.org/conformance-explorer/>. Accessed: 2026-03-24.
- [27] C2PA, “C2PA Conformance Program Public Repository.” <https://github.com/c2pa-org/conformance-public>.
- [28] N Krawetz, “c2pa-retimestamp.” <https://github.com/hackerfactor/c2pa-retimestamp>. Accessed: 2026-03-26.
- [29] A. Horshack, “My Z6 III C2PA deep dive.” <https://tinyurl.com/4vezjyzj>, September 2025. Accessed: 2026-03-25.
- [30] C2PA, “Conformance.” <https://c2pa.org/conformance/>. Accessed: 2026-03-26.
- [31] Adobe, “verify-site Commits.” <https://github.com/contentauth/verify-site/commits/main/static/trust/allowed.pem>. Accessed: 2026-03-26.
- [32] R. Zuccherato, P. Cain, D. C. Adams, and D. Pinkas, “Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP).” RFC 3161, Aug. 2001.
- [33] Information Technology & Innovation Foundation (ITIF), “Context matters: Building trust in digital content.” <https://itif.org/events/2026/03/05/context-matters-building-trust-in-digital-content/>, March 5, 2026. Panel presentation to congressional staff, lobbyists, and policymakers.
- [34] D. Dolev and A. Yao, “On the security of public key protocols,” *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.
- [35] M. Liskov, P. Rowe, and J. Thayer, “Completeness of CPSA,” tech. rep., MITRE, 2011. https://www.mitre.org/sites/default/files/pdf/12_0038.pdf.
- [36] N. Admin, “Nikon sent out a new notice informing users that all C2PA certificates issued will be revoked,” *Nikon Rumors*, Sept 2025. Accessed: 2026-01-13.
- [37] Coalition for Content Provenance and Authenticity, “C2PA Technical Specification, Version 2.3.” <https://spec.c2pa.org/specifications/specifications/2.3/index.html>. C2PA Specifications. Accessed: 2026-01-12.
- [38] M. Looram, “How the Times assessed that photo from Trump of Maduro in handcuffs,” *The New York Times Company*, 1 2026.
- [39] J. Lukáš, J. Fridrich, and M. Goljan, “Digital camera identification from sensor pattern noise,” *IEEE Transactions on Information Forensics and Security*, 2006.
- [40] M. Stamm and K. Liu, “Forensic detection of image manipulation using statistical intrinsic fingerprints,” *IEEE Transactions on Information Forensics and Security*, 2010.
- [41] J. Weatherbed, “Benjamin Netanyahu is struggling to prove he’s not an AI clone,” *The Verge*, March 2026.
- [42] M. L. Psiaki and T. E. Humphreys, “GNSS spoofing and detection,” *Proceedings of the IEEE*, vol. 104, no. 6, pp. 1258–1270, 2016.
- [43] J. Bushey, N. Rivard, and M. Barbeau, “Cryptographic provenance and AI-generated images.” https://ai-collaboratory.net/wp-content/uploads/2025/11/S13212_7356.pdf, 2025.
- [44] N. Earnshaw, J. Dupras, and B. MacCormack, “Fighting Misinformation with Authenticated C2PA Provenance Metadata,” in *Proceedings of the 2023 NAB BEIT Conference*, 2023.

- [45] N. Krawetz, "Authenticity from the attacker's perspective (C2PA from the attacker's perspective)." Slides (PDF), IPTC Photo Metadata Conference, May 2024.
- [46] N. Krawetz, "Shmocon and C2PA forgeries." <https://hackerfactor.com/blog/index.php/?archives/1058-Shmocon-and-C2PA-Forgeries.html>. Accessed: 2026-01-13.
- [47] P. Fisher, "Preserving media provenance metadata with C2PA." <https://www.eidr.org/assets/Preserving-Media-Provenance-Metadata-with-C2PA.pdf>, 2025.
- [48] K. Kaye and P. Dixon, "Privacy, Identity and Trust in C2PA," *World Privacy forum*, 2025. Accessed: 2026-01-14.
- [49] N. Patel, "Reality is losing the deepfake war: Why AI labeling efforts are falling flat in the face of slop, disinformation, and messy metadata standards," *The Verge*, February 2026.
- [50] S. Petrangeli, H. Wang, M. Fisher, D. Kozma, M. Mahamli, P. Blumenthal, and A. Parsons, "Integrating content authenticity with DASH video streaming," in *Proceedings of the 15th ACM Multimedia Systems Conference, MMSys 2024, Bari, Italy, April 15-18, 2024*, pp. 492–498, ACM, 2024.
- [51] N. Fotos and J. Delgado, "Ensuring privacy in provenance information for images," in *2023 24th International Conference on Digital Signal Processing (DSP)*, pp. 1–5, IEEE, 2023.
- [52] P. Della Monica, I. Visconti, A. Vitaletti, and M. Zecchini, "Trust nobody: Privacy-preserving proofs for edited photos with your laptop," in *2025 IEEE Symposium on Security and Privacy (SP)*, pp. 4624–4642, IEEE, 2025.
- [53] S. Park, A. Vilesov, J. Zhang, H. Khalili, Y. Tian, A. Kadambi, and N. Sehatabkhsh, "Chimera: Creating digitally signed fake photos by fooling image recapture and deepfake detectors," in *Proceedings of the 34th USENIX Security Symposium (USENIX Security 2025)*, 2025.
- [54] J. Young, S. Vaughan, A. Jenks, H. Malvar, C. Paquin, P. England, T. Roca, J. Ferres, F. Poursabzi, N. Coles, K. Archer, and E. Horvitz, "Media integrity and authentication: Status, directions, and futures." https://www.microsoft.com/en-us/research/wp-content/uploads/2026/02/Media-Integrity_Authentication-Report_Microsoft_021926.pdf. Accessed: 2026-04-23.
- [55] International Press Telecommunications Council, "IPTC Digital Source Enumeration." <https://schema.org/IPTCDigitalSourceEnumeration>. Schema.org. Accessed: 2026-01-12.
- [56] Adobe, "XMP." <https://www.adobe.com/products/xmp/standards.html>. Accessed: 2026-01-12.
- [57] Google DeepMind, "SynthID." <https://deepmind.google/models/synthid/>. Accessed: 2026-01-12.
- [58] Adobe, "TrustMark." <https://github.com/adobe/trustmark/>. Accessed: 2026-01-12.
- [59] Meta, "Stable Signature: A new method for watermarking images created by open source generative AI." <https://ai.meta.com/blog/stable-signature-watermarking-generative-ai/>. Accessed: 2026-01-12.
- [60] Hacker Factor, "Secure Evidence Attribution Label (SEAL)." <https://github.com/hackerfactor/SEAL>. Accessed: 2026-01-12.
- [61] Numbers Protocol, "Numbers Protocol documentation." <https://docs.numbersprotocol.io/introduction/>. Accessed: 2026-01-12.
- [62] Starling Lab, "The Starling Lab framework." <https://ffdw.org/blog/the-starling-lab-framework>. Accessed: 2026-01-12.
- [63] Creator Assertions Working Group, "About CAWG." <https://cawg.io/about/>. Accessed: 2026-01-13.
- [64] World Wide Web Consortium, "Verified Credentials Overview." <https://www.w3.org/TR/vc-overview/>. Accessed: 2026-04-07.
- [65] Adobe, "c2pa-js." <https://github.com/contentauth/c2pa-js>. Accessed: 2026-01-16.
- [66] S. Santesson, M. Myers, R. Ankey, A. Malpani, S. Galperin, and C. Adams, "X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP." RFC 6960, June 2013.
- [67] Google Pixel Support Team, "Google Pixel Update - December 2025." <https://support.google.com/pixelphone/thread/389367100/google-pixel-update-december-2025>. Accessed: 2026-01-15.
- [68] Adobe, Hacker Factor, "c2patool segfaults on altered manifest #1578." <https://github.com/contentauth/c2pa-rs/issues/1578>. Accessed: 2025-12-29.
- [69] Coalition for Content Provenance and Authenticity, "C2PA Technical Specification, Version 2.4." <https://spec.c2pa.org/specifications/specifications/2.4/index.html>. C2PA Specifications. Accessed: 2026-05-21.
- [70] Office of the Chief Information Officer, "Official media created and certified by Arizona's Secretary of State Office." <https://acm.azsos.gov/>.
- [71] S. Lowenstein, "Using secure sourcing to combat misinformation." <https://rd.nytimes.com/projects/using-secure-sourcing-to-combat-misinformation/>. Accessed: 2025-12-29.
- [72] S. Labs, "Image authentication." <https://www.starlinglab.org/image-authentication/>. Accessed: 2021-04-23 and 2026-03-29.
- [73] S. Shivji, "India's hindu nationalists are petitioning courts to tear down mosques and replace them with temples." <https://www.cbc.ca/news/world/hindu-nationalists-mosques-india-1.7193591>. Accessed: 2024-08-09 and 2026-03-29.
- [74] Andy Parsons, "Authenticity in the age of AI: Growing content credentials momentum across social media platforms, AI companies and rising consumer awareness." <https://blog.adobe.com/en/publications/2024/09/18/authenticity-age-ai-growing-content-credentials-momentum-across-social-media-platforms-ai-companies-rising-consumer-awareness>. Accessed: 2026-02-12.
- [75] Adobe, "Media Alert: Adobe Introduces Adobe Content Authenticity Web App to Champion Creator Protection and Attribution." <https://news.adobe.com/news/2024/10/aca-announcement>. Accessed: 2026-03-29.
- [76] US Government, "52 USC 20701: Retention and preservation of records and papers by officers of elections; deposit with custodian; penalty for violation." <https://uscode.house.gov/view.xhtml?req=granuleid:USC-prelim-title52-section20701&num=0&edition=prelim>.
- [77] National Archives and Records Administration, "System architecture and design document (SADD) summary, Non-CDRL document, Version 1.1.0." <https://www.archives.gov/files/foia/pdf/system-arch-design-29sept2006.pdf>.
- [78] E. Golaszewski, A. T. Sherman, E. Ziegler, J. D. Fuchs, and S. Hamer, "Cryptographic binding should not be optional: A formal-methods analysis of FIDO UAF channel binding," in *Proceedings of Security Standardisation (SSR) 2025*, Springer, 2025. Long version available as arXiv preprint <http://arxiv.org/abs/2511.06028>.
- [79] D. E. 3rd, "The Transport Layer Security (TLS) Multiple Certificate Status Request Extension." RFC 6066, Jan. 2011.
- [80] E. Horvitz, "A promising step forward on disinformation." <https://blogs.microsoft.com/on-the-issues/2021/02/22/deepfakes-disinformation-c2pa-origin-cai/>. Accessed: 2026-01-13.
- [81] C2PA, "C2PA Founding Press Release." <https://c2pa.org/c2pa-founding-press-release/>. Accessed: 2026-01-13.
- [82] A. R. Locker, C. Heitzenrater, and T. Helmus, "Overpromising on digital provenance and security." <https://www.rand.org/pubs/commentary/2025/06/overpromising-on-digital-provenance-and-security.html>. Accessed: 2026-01-13.
- [83] N. Krawetz, "C2PA's Worst Case Scenario." <https://hackerfactor.com/blog/index.php/?archives/1013-C2PAs-Worst-Case-Scenario.html>. Accessed: 2026-01-13.
- [84] N. Krawetz, "The Big Bulleted List." <https://hackerfactor.com/blog/index.php/?archives/1069-The-Big-Bulleted-List.html>. Accessed: 2026-03-31.
- [85] Project Origin, "Protecting trusted media overview." <https://web.archive.org/web/20201204055201/https://www.originproject.info/about>. Accessed: 2026-01-13.

[86] A. Jenks, “Re: NIST AI 100- 4, Reducing risks posed by synthetic content: An overview of technical approaches to digital content transparency.” https://downloads.regulations.gov/NIST-2024-0001-0030/attachment_1.pdf. Accessed: 2026-01-13.

Appendix A. Open Science

All of our artifacts are available on GitHub [11], including our CPSA input models and output shapes.

Appendix B. Ethical Considerations

We have responsibly disclosed our findings to the C2PA Steering Committee via Adobe and Google, by exchanging emails and sharing preliminary drafts of this paper. Representatives from Adobe and Google attended many of our biweekly meetings. Based on our inputs, the C2PA specifications—including Version 2.3—have dropped some unsubstantiated claims and implemented some improvements.

Appendix C. Acronyms and Abbreviations

AI	artificial intelligence
API	application programming interface
C2PA	Coalition for Content Provenance and Authenticity
CA	certificate authority
CAI	Content Authenticity Initiative
CAWG	Creator Assertions Working Group
CPSA	Cryptographic Protocol Shapes Analyzer
Cr	content credentials
CRL	certificate revocation list
DRM	digital rights management
DY	Dolev-Yao
EXIF	Exchangeable Image File Format
IPTC	International Press Telecommunications Council
ITL	interim trust list
JPEG	Joint Photographic Experts Group
JUMBF	JPEG Universal Metadata Box Format
OCSP	Online Certificate Status Protocol
PKI	public-key infrastructure
PNG	Portable Network Graphics
TEM	trusted execution module
TPM	trusted platform module
TSA	trusted timestamping authority
TST	trusted timestamp
UMBC	University of Maryland, Baltimore County
URI	uniform resource identifier
URL	uniform resource locator
WASM	WebAssembly
XMP	Extensible Metadata Platform

Appendix D. Ladder Diagrams

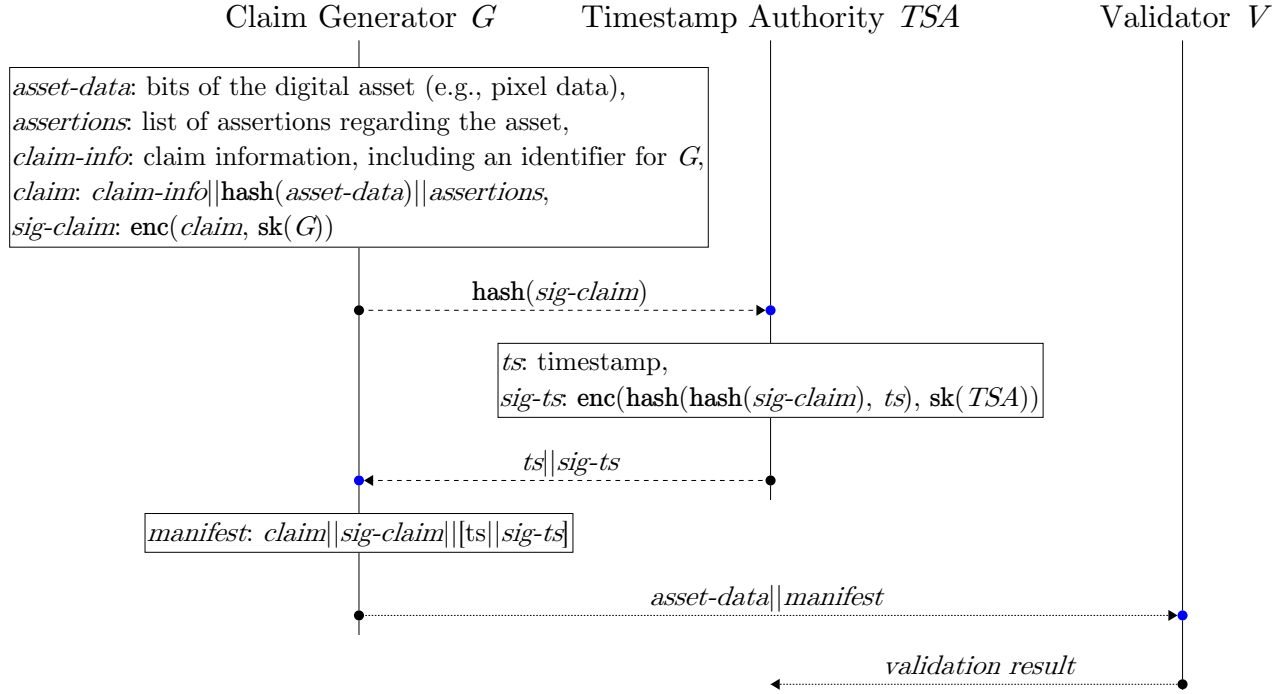


Figure 6. Ladder diagram of the core C2PA protocol. Vertical lines correspond to the communicating roles (G, TSA, V). Arrows between vertical lines indicate the transmission and reception of messages. Arrows with *dashed lines* indicate optional messages for generating the claim-signature timestamp $sig-ts$, which the protocol encourages but does not require. When G does not timestamp the claim signature, *manifest* omits the signed timestamp $sig-ts$. Arrows with *dotted lines* indicate messages that transmit indirectly through users on the Internet.

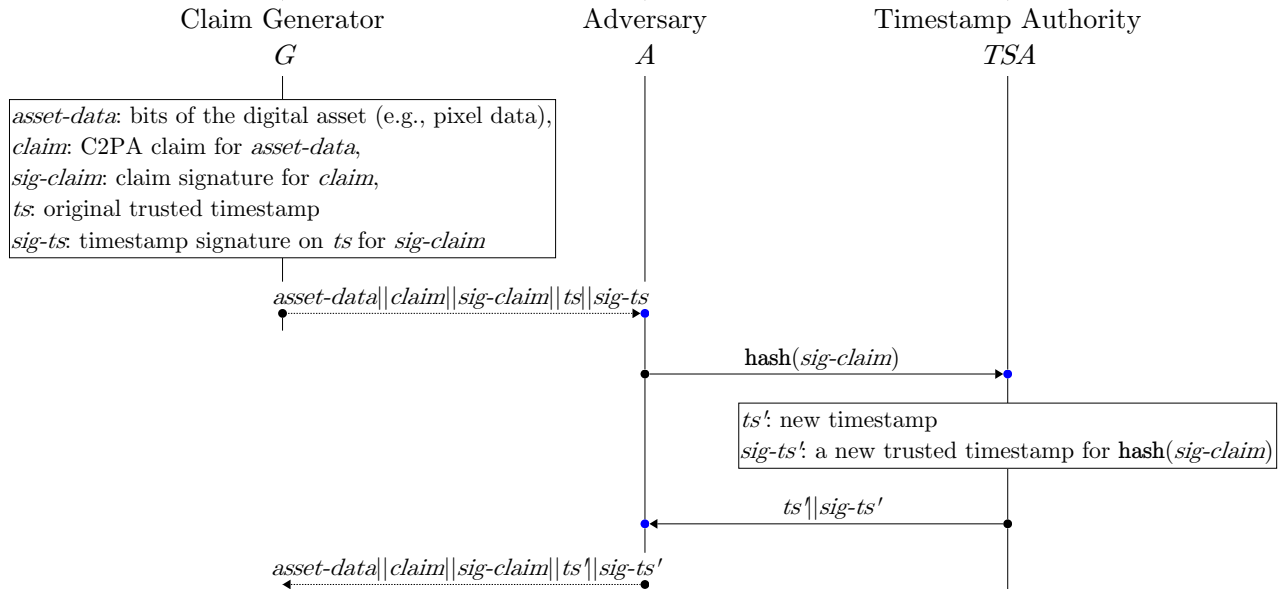


Figure 7. Ladder diagram illustrating an attack on C2PA's RFC3161 timestamp. Because G fails to bind the timestamp signature $sig-ts$ to the claim, an adversary A communicates with a legitimate timestamping authority TSA to substitute $sig-ts$ for an alternate, malicious (or blank) $sig-ts'$. The specifications permit this scenario because it allows G to generate a claim, then request and append a timestamp for the claim at a later time.

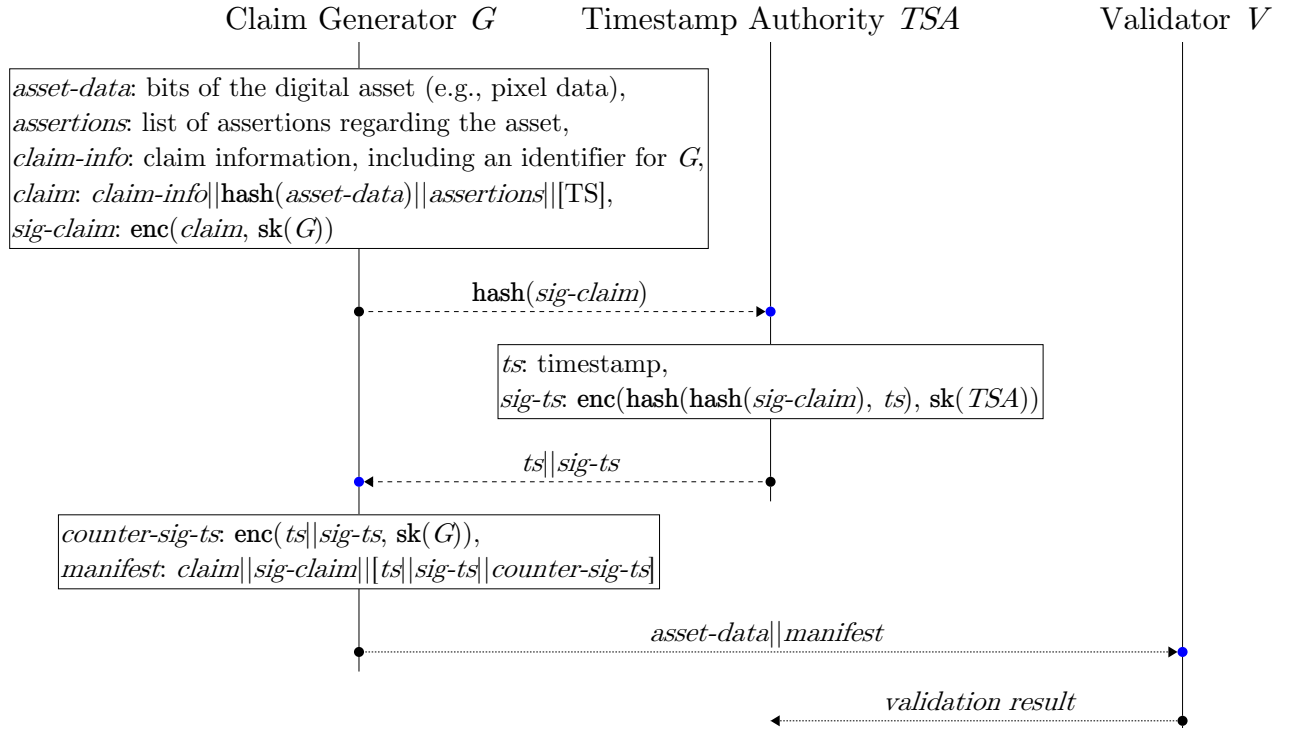


Figure 8. Ladder diagram of our main recommended improved C2PA protocol, which we call *C2PA-GeneratorSignature* (Section 7). This improvement makes two changes to the core C2PA protocol: First, we include an assertion TS in the claim, which promises that a trusted timestamp will accompany the claim signature. Second, we include a counter signature $counter-sig-ts$ in the claim $manifest$: this addition prevents an adversary from substituting or eliminating $sig-ts$ from $manifest$. Although no implementations that we are aware of do so, the specification defines timestamp assertions, which exist within the claim. Incorporating the timestamp into the claim’s assertions eliminates the need for $counter-sig-ts$. In both scenarios, a manifest receiving a timestamp at a later time must promise the timestamp using the TS field. If TS is absent, the validator should treat any present timestamp as an error.

Appendix E.

Formal-Methods Analysis

We perform a formal-methods analysis of the C2PA protocol against security goals SG1, SG2, SG3, and SG5, which promise data authentication for the claims in a C2PA manifest, data authentication for optional timestamps, and data integrity of the asset (Section 5). We do so by applying the strand-space formalism. This analysis involves verifying that the origin and integrity of claims associated with a digital asset have not been altered and that they come from a legitimate source. We carry out our analysis for each of the five protocol models in Table 2, referring to them by their model index $i \in \{0, 1, 2, 3, 4\}$. Our proposed improvements are Models 2 and 4.

For the purposes of this analysis, a legitimate source is simply an entity with an uncompromised signing key. We also verify data authentication for optional timestamps by checking that each timestamp includes the trusted timestamp authority that created the timestamp and the source that included the timestamp with the asset's manifest. We perform our analysis wishfully assuming that the public-key certificates are valid.

To carry out our analysis, we (1) define a *strand space* (Definition E.1) for the C2PA protocol, (2) formalize the C2PA security goals as logical formulas describing properties of distinct execution models in the C2PA strand space, and (3) use the *Cryptographic Protocol Shapes Analyzer* (CPSA) [10] to verify the formal security goals. Our CPSA inputs and outputs are available on GitHub [11].

To describe the strand space, it is useful to introduce the concept of a *trace*, which is a finite, non-empty sequence of events (message transmissions or receptions) that take place within a designated channel.

Definition E.1 (Strand Space). Let A be the set of all possible protocol terms, and d is $-$ or $+$, indicating reception or transmission. A *directed term* is a pair (d, t) , where $t \in A$. A *strand* s comprises a trace of directed terms, where s specifies the actions of a legitimate party or of the adversary. A *strand space* is a set, Σ , of strands.

E.1. C2PA Strand Space

To define the C2PA strand space (Definition E.13), we define *penetrator strands* that model adversarial behavior and *regular strands* that model behavior of legitimate protocol roles. Definition E.2 specifies the set Σ_{pen} of penetrator strands. Definition E.12 specifies the set of regular strands that carry out the roles of the claim generators Σ_g , the trusted timestamp authorities Σ_T , and the validators Σ_V .

We use the C2PA strand space, which combines the penetrator and regular strands, to model protocol executions that potentially incorporate adversarial behavior.

Definition E.2 (Penetrator Strands). Let $T \subseteq A$ be a set of atoms that represent principals, string literals, keys, and nonces. Let $K \subseteq T$ be a set of all encryption keys, including

inverse keys. Let $e(t, k)$ be the encryption of term $t \in A$ under key $k \in K$. Depending on whether k is a symmetric or asymmetric key, $e(t, k)$ may denote either symmetric or asymmetric encryption; k is a symmetric key when $k^{-1} = k$. Let $h(\cdot)$ be a perfect hash function, where $h(t)$ represents the hash of term t . The set Σ_{pen} is a set of all strands $p \in \Sigma$, where p has one of the penetrator traces below:

- 1) *Generate*(t): $[+t]$ where $t \in T$.
- 2) *Hash*(g): $[-g, +h(g)]$ where $g \in A$.
- 3) *Concatenate*(g, h): $[-g, -h, +g||h]$ where $g, h \in A$.
- 4) *Separate*($g||h$): $[-g||h, +g, +h]$ where $g, h \in A$.
- 5) *Encrypt*(g, k): $[-g, -k, +e(g, k)]$ where $g \in A, k \in K$.
- 6) *Decrypt*($e(g, k), k^{-1}$): $[-e(g, k), -k^{-1}, +g]$ where $g \in A, k \in K, k^{-1} \in K$.

C2PA relies on cryptographic signatures to authenticate the origin of data, the claims created, and timestamps generated. In Definition E.3, the function pk expresses a principal's public key, and sk expresses the corresponding private key.

Definition E.3 (Public and Private Keys). Let $pk : Name \rightarrow K$ and $sk : Name \rightarrow K$ be one-to-one functions with disjoint images such that $pk(a) = sk(a)^{-1}$ for all $a \in Name$.

The exchanges that C2PA uses consist of three fundamental subsets of A . *Name* is the set of parties communicating in the protocol. *Data* is the set of images to which provenance data should be attached, originating at a generator. *Text* is the set of all supporting text included with an image; it is the assertions associated with a given image. We use the abbreviations *at* for asset, *an* for assertion, and *S* for an arbitrary signer of data. We now define the C2PA claim signature (Definition E.4) and the C2PA claim (Definition E.5).

Definition E.4 (Claim Signature). Let $S \in Name$, $at \in Data$, $an \in Text$. We define the Claim Signature as:

$$\text{Sig}_S(at, an) := e(h(an||h(at)), sk(S)).$$

Definition E.5 (Claim). Let $S \in Name$, $at \in Data$, $an \in Text$. We define the claim as:

$$\text{Claim}_S(at, an) := an||\text{Sig}_S(at, an).$$

We provide variants of the claim signature (Definition E.6) and claim (Definition E.7) to describe the strands we have modified in our efforts to prove SG3. These modifications will be discussed further in our analysis (Section E.3).

Definition E.6 (Claim Signature (fix)). Let $S \in Name$, $at \in Data$, $an, Flag \in Text$. We define the Claim Signature as:

$$\text{Sig}_S^f(at, an) := e(h(an||h(at)||Flag), sk(S)).$$

Definition E.7 (Claim (fix)). Let $S \in Name$, $at \in Data$, $an \in Text$. We define the claim as:

$$\text{Claim}_S^f(at, an) := an||\text{Sig}_S^f(at, an).$$

C2PA supports the addition of timestamps from RFC 3161 compliant trusted *timestamping authorities* (TSA).

When a timestamp is requested from a TSA, the requester provides a value representing the hash of the message on which it wishes to receive a timestamp. The TSA returns the timestamp including the value, the time, and a signature over the value and time using the TSA's private key to certify authentication. We refer to this timestamp as the TST.

Definition E.8 (Signed Timestamp). Let $ts \in Text$, $val \in A$, and $TSA \in Name$. Define the Signed Timestamp as:

$$TST_{TSA}(val) := val || ts || e(h(val || ts), sk(TSA)).$$

We define strands for legitimate participants in the protocol, the Generator, TSA, and the Validator. For the Generator and Validator, there are five different traces that can produce a strand. The Generator traces represent the two possible traces of the C2PA protocol as specified, with and without a timestamp request, and the three traces for proposed modifications that could allow the security goals SG1, SG2, SG3, and SG5 to be proven. Each of the Validator traces represents a Validator corresponding to the final message produced by each Generator.

Definition E.9 (Generator Strands). Let $G, TSA \in Name$ represent the names of generators and TSAs, $at \in Data$ represent images, and $an, ts \in Text$ represent assertions and timestamps. The set Σ_G is the set of all strands $p \in \Sigma$, where p has one of the generator traces below:

- 1) $Tr_{G0}(G, at, an) := [+at || Claim_G(at, an)].$
- 2) $Tr_{G1}(G, TSA, at, an) := [+h(Sig_G(at, an)), -TST_{TSA}(h(Sig_G(at, an))), +at || Claim_G(at, an) || TST_{TSA}(h(Sig_G(at, an)))].$
- 3) $Tr_{G2}(G, TSA, at, an) := [+h(Sig_G^f(at, an)), -TST_{TSA}(h(Sig_G^f(at, an))), +at || Claim_G^f(at, an) || TST_{TSA}(h(Sig_G^f(at, an))) || e(TST_{TSA}(h(Sig_G^f(at, an))), sk(G))].$
- 4) $Tr_{G3}(G, TSA, at, an) := [+h(Sig_G(at, an)), -TST_{TSA}(h(Sig_G(at, an))), +at || Claim_G(at, an) || Claim_G(at, TST_{TSA}(h(Sig_G(at, an)))) || Claim_G(at, an)].$
- 5) $Tr_{G4}(G, TSA, at, an) := [+h(Sig_G^f(at, an)), -TST_{TSA}(h(Sig_G^f(at, an))), +at || Claim_G^f(at, an) || Claim_G(at, TST_{TSA}(h(Sig_G^f(at, an)))) || Claim_G^f(at, an)].$

Definition E.10 (Timestamp Authority Strands). Let $TSA \in Name$ represent the names of TSAs and $m \in A$ represent messages to be timestamped. The set Σ_T is the set of all strands $p \in \Sigma$ where p has the following trace: $[-m, +TST_{TSA}(m)].$

Definition E.11 (Validator Strands). Let $G, TSA \in Name$ represent the names of generators and TSAs, $at \in Data$ represent images, and $an, ts \in Text$ represent assertions and timestamps. The set Σ_V is the set of all strands $p \in \Sigma$ where p has one of the validator traces below:

- 1) $Tr_{V0}(G, at, an) := [-at || Claim_G(at, an)].$

- 2) $Tr_{V1}(G, TSA, at, an) := [-at || Claim(at, an) || TST_{TSA}(h(Sig_G(at, an)))].$
- 3) $Tr_{V2}(G, TSA, at, an) := [-at || Claim^f(at, an) || TST_{TSA}(h(Sig_G^f(at, an))) || e(TST_{TSA}(h(Sig_G^f(at, an))), sk(G))].$
- 4) $Tr_{V3}(G, TSA, at, an) := [-at || Claim_G(at, an) || Claim_G(at, TST_{TSA}(h(Sig_G(at, an)))) || Claim_G(at, an)].$
- 5) $Tr_{V4}(G, TSA, at, an) := [-at || Claim_G^f(at, an) || Claim_G(at, TST_{TSA}(h(Sig_G^f(at, an)))) || Claim_G^f(at, an)].$

Definition E.12 (C2PA Regular Strands). The regular strands of C2PA comprise $\Sigma_{reg} = \Sigma_G \cup \Sigma_T \cup \Sigma_V$.

Definition E.13 (C2PA Strand Space). The C2PA strand space Σ_{C2PA} comprises $\Sigma_{pen} \cup \Sigma_{reg}$.

E.2. Formal Security Goals

We restate the four security goals from Section 5 that we analyze within the C2PA strand space Σ_{C2PA} as agreement propositions. Integrity and authentication of origin can be formalized as agreement between validator strands and generator strands over sets of variables representing assets, claims, timestamps, and identities under origination assumptions governing the secrecy of certain keys. We begin by formally defining agreement (Definition E.14) within a Strand Space Σ .

Definition E.14 (Agreement). For a strand space Σ of a given protocol, let $\Sigma_{reg} = \Sigma - \Sigma_{pen}$ represent the set of regular strands in the strand space of the protocol (Definition E.12). Let $\Sigma_X, \Sigma_Y \subset \Sigma_{reg}$, where $\Sigma_X \cap \Sigma_Y = \emptyset$, represent disjoint sets of regular strands consisting of traces of directed terms Tr_X and Tr_Y , respectively. Let $s_X \in \Sigma_X$ and $s_Y \in \Sigma_Y$ be regular strands in their respective strand spaces. Let $\vec{C}$ be any list of terms $t \in A$ for which to test agreement. Let **orig** represent a conjunction of origination predicates. Then,

$$Agree_{\Sigma}(s_X, s_Y, \vec{C}, \mathbf{orig}) \stackrel{def}{\iff} ((\forall s_X \in \Sigma_X, \vec{C} \subseteq s_X) \wedge \mathbf{orig}) \implies (\exists s_Y \in \Sigma_Y, \vec{C} \subseteq s_Y). \quad (1)$$

C2PA relies on the secrecy of signing keys to provide the integrity and authentication of manifests. Within the strand-space formalism, these secrecy properties are specified as non-origination. A *non-originating* term within a strand space is a term that is unknown to the adversary, does not occur in any penetrator strands, and is never carried in any message of a regular strand. It can never be acquired by a penetrator strand. A term $t \in A$ is specified as non-originating as $Non(t)$.

The security goals are specified as three propositions on agreement. Proposition 1, *Data Agreement 1*, addresses security goals SG1, SG2, and SG5. It is defined as agreement between generator and validator strands on the asset, assertions, and identity of the generator of the assertions.

Proposition 1 (Data Agreement 1). *For all strands $z \in \Sigma_V$, $G \in \text{Name}$, $at \in \text{Data}$, and $an \in \text{Text}$, where G , at , an occur in the trace of z , implies there exists a $z_0 \in \Sigma_G$, $\text{Agree}_\Sigma(z, z_0, (G, at, an), \text{Non}(sk(G)))$.*

To address security goal SG3, we need two propositions. The first, Proposition 2, *Data Agreement 2*, is a minor variation of Proposition 1 to include a timestamp and the trusted timestamp authority that produced it. It is defined over the subset of Σ_V that receives timestamps. We define disjoint subsets of Σ_V by the traces. For example, the set Σ_{V0} is the set of all strands $p \in \Sigma_V$, where p has the trace Tr_{V0} . The other subsets are defined similarly, such that $\Sigma_{V0} \cup \Sigma_{V1} \cup \Sigma_{V2} \cup \Sigma_{V3} \cup \Sigma_{V4} = \Sigma_V$. We exclude Σ_{V0} from Proposition 2 because it does not receive a message with a timestamp and cannot satisfy SG3. The proposition specifies that, if a validator receives a timestamp from a trusted timestamp authority with an asset and manifest from a generator, the generator agrees that it provided the information.

Proposition 2 (Data Agreement 2). *For all strands $z \in \Sigma_{Vi}$ for each $i \in \{1, 2, 3, 4\}$, $G, TSA \in \text{Name}$, $at \in \text{Data}$, and $an, ts \in \text{Text}$, where G, TSA, at, an, ts occur in the trace of z , implies there exists a $z_0 \in \Sigma_G$, such that*

$$\text{Agree}_\Sigma(z, z_0, (G, TSA, at, an, ts), (\text{Non}(sk(G)), \text{Non}(sk(TSA)))).$$

The second proposition necessary to satisfy SG3, Proposition 3 *Fixed Timestamp*, states that validators that do not receive a timestamp cannot satisfy *Agreement* (Definition E.14) with a generator that provides a timestamp. This proposition prevents the TSA and timestamp from being removed from the terms $\vec{C}$ on which to test for agreement if a generator provides a timestamp.

Proposition 3 (Fixed Timestamp). *For all strands $z \in \Sigma_{V0}$, $G \in \text{Name}$, $at \in \text{Data}$, and $an \in \text{Text}$, where G, at, an occur in the trace of z , implies there does **not** exist a $z_0 \in \Sigma_G - \Sigma_{G0}$ such that $\text{Agree}_\Sigma(z, z_0, (G, at, an), \text{Non}(sk(G)))$.*

We discovered the need for this proposition in our analysis of Proposition 1 (see Section E.3). We found that there exist generator strands that provide timestamps that would satisfy *Agreement* (Definition E.14) with validator strands that did not receive the timestamps. These strands would allow generator provided timestamps to be undetectably removed, invalidating agreement for SG3.

E.3. Analysis

To perform our analysis, we use CPSA. CPSA takes a model of the protocol within the strand space theory and a “skeleton,” a partial execution of the protocol with security assumptions. It explores all possible executions of the protocol that satisfy the skeleton in the presence of a DY adversary. If CPSA terminates, it outputs a complete characterization of all essentially different classes of execution

that satisfy the skeleton as “shapes”. From these shapes, the corresponding security goals and corresponding proofs can be derived.

The security goals SG1, SG2, and SG5 are defined as *Agreement* (Theorem 1) on the asset, assertions, and identity of the originator between generator and validator strands as expressed in Proposition 1. We do not explicitly distinguish between SG2 and SG5 in our models, because the only difference is that not all of the asset’s bits are protected by a signature in SG2, leaving those bits available for manipulation. Instead, we treat the asset as if it were entirely covered by the signature.

Theorem 1 (Data Agreement 1). *Proposition 1 is true.*

Proof by Enumeration. Carrying out an exhaustive search, CPSA enumerates all essentially different (unique) shapes for regular strands in Σ_V under *Proposition 1*. The security goal *Agreement* as specified in *Proposition 1* holds under all shapes. $\square$

Concerning our CPSA analysis of *Theorem 1*, we note three points. *First*, because there are several different messages that validators can receive based on different generator traces (C2PA’s and our proposed fixes) and CPSA does not support branching, it is necessary to evaluate the theorem for each of the possible messages. To do so, we subset Σ_V and Σ_G by their traces, as discussed in Section E.2, and verify *Data Agreement* by evaluating the goal under each subset of Σ_V against the entire set of Σ_G . Within the expressed security goal, we define the set of Σ_G as a disjunction of the subsets of Σ_G .

Second, the evaluation of *Theorem 1* for validator strands in Σ_{V0} include shapes with generator strands from Σ_{G1} and Σ_{G3} in addition to a shape with strands from Σ_{G0} , the expected shape. Although these additional shapes satisfy *Theorem 1*, they illustrate two issues with C2PA. The shape containing a generator strand from Σ_{G1} illustrates that a trusted timestamp can be removed from a manifest and the manifest is still valid. As trusted timestamps are added after the manifest has been signed, there is nothing in a C2PA manifest to indicate that a timestamp should be present. This result led us to realize that *Proposition 2* would be insufficient to satisfy SG3, as there would be shapes that are possible that are not covered by *Proposition 2*. *Proposition 2* does not include strands from Σ_{V0} , for which shapes should not exist.

The shape with strands from Σ_{G3} is one of our failed proposed fixes to protect the timestamp by including it in an additional manifest as a signed claim. It illustrates that manifests can be removed from an asset. In general, if a manifest is not available, there is nothing to validate, but in this case it demonstrates that manifests can be removed without affecting the validation of other manifests, provided those manifests do not reference the removed manifests.

The shapes that illustrate the removal of timestamps from generator strands with traces of Tr_{G1} and Tr_{G3} also point to another problem with the use of C2PA’s optional timestamp (Model 1), and the use of a manifest to store the

timestamp as a claim (Model 3). SG3 cannot be satisfied, as the validators are unable to verify the missing timestamp information provided by the generators or even identify that the intended information is missing.

Third, a shape is produced in the analysis of the validator strands defined by Tr_{V1} . This trace includes trusted timestamps as defined by C2PA, with a strand from a generator trace Tr_{G0} , a generator strand that does not add trusted timestamps. This fact illustrates that timestamps can be added to assets by anyone. Example 1 (Section 3.1) demonstrates the ability to replace timestamps by removing the original timestamp and adding a new timestamp.

The security goal SG3 is defined as *Agreement* on the asset, assertions, timestamp, TSA, and identity of the originator between generator and validator strands when a generator provides a timestamp. Proposition 2 addresses agreement on the set of terms to satisfy SG3. Proposition 3 provides verification that when a generator provides a timestamp, the set of terms on which *Agreement* is reached includes the timestamp and TSA. SG3 is satisfied if both Proposition 2 and Proposition 3 are true.

We attempt to prove SG3 for each of the possible protocol models with timestamps, the C2PA specified execution, and our proposed fixes. We do so by verifying whether or not Proposition 2 and Proposition 3 hold for each model. The protocol models are defined by their corresponding traces. For example, Model 0 is defined by Generator Strands (Definition E.9) with trace Tr_{G0} and Validator Strands (Definition E.11) with trace Tr_{V0} . Using CPSA we proved the following theorems for Proposition 2.

Theorem 2 (Data Agreement 2 for Σ_{V1}). *Proposition 2 is false.*

Proof by Counterexample. Using CPSA, we identify all unique shapes that contradict Proposition 2. In each shape, a legitimate strand of Σ_{V1} fails to agree on the values of *TSA* and *ts* with any regular strand of Σ_G . $\square$

Theorem 3 (Data Agreement 2 for Σ_{Vi} for $i \in \{2, 3, 4\}$). *Proposition 2 is true.*

Proof by Enumeration. Carrying out an exhaustive search, CPSA enumerates all essentially different (unique) shapes for regular strands in Σ_{V2} , Σ_{V3} , and Σ_{V4} under Proposition 2. The security goal *Agreement* specified in Proposition 2 holds under all shapes. $\square$

From Theorem 2, we can see that Model 1, the C2PA specified timestamp protocol fails to achieve *Agreement* on the timestamp and TSA. CPSA output indicates that it is possible for an adversary to swap the timestamp associated with an asset by a generator. Example 1 (Section 3.1) demonstrates this vulnerability. All three of our proposed fixes, which require the generator to provide a signature on the timestamp satisfy Proposition 2, as can be seen from Theorem 3.

To determine which models satisfy SG3, it is also necessary to verify Proposition 3. For the proposition to be satisfied, CPSA should not produce shapes. Any shape that

is produced would indicate that there exists a generator strand that produces a timestamp for which validators do not achieve *Agreement* on the timestamp and TSA. CPSA produced two such shapes for generator strands from Σ_{G1} and Σ_{G3} . These shapes indicate that Model 1, which we had already determined did not satisfy SG3, and Model 3, which simply added the signed timestamp manifest, fail to satisfy SG3. No shapes were found for generator strands from Σ_{G2} and Σ_{G4} indicating that Model 2 and Model 4 satisfied Proposition 3. By satisfying both Proposition 2 and Proposition 3, Model 2 and Model 4 satisfied SG3.

From our formal analysis, assuming that all public-key certificates are valid, we have demonstrated that the protocol meets security goals SG1 and SG2. However, we have shown that by not requiring the generator to sign timestamps attached to manifests, C2PA as currently specified cannot satisfy the security goal SG3. We have also demonstrated that signing alone (Model 3) is insufficient to satisfy SG3. Our effective proposed fixes include a flag to indicate that a timestamp should be present in the claim signature to prevent the timestamp from being undetectably removed, allowing SG3 to be satisfied. The security goal SG5 can be satisfied if no part of the asset is excluded from verification of the asset's integrity. Our models demonstrate this condition by including the entire asset in the signature.

Appendix F. Additional Examples

We present three additional examples that demonstrate more limitations, vulnerabilities, and attacks on C2PA based on deficiencies in the specifications.

Example 4: Pre-Capture Manipulation



```
Asset: Manifest:urn:c2pa:05a989c3...
1 Assertion:c2pa.actions.v2
  Action: "c2pa.created"
  Description:
    "Created by Pixel Camera."
2 Assertion:c2pa.hash.data
3 Signature:
  Issuer: "Google LLC"
  Time: "2025-10-04T03:48:33+00:00"
```

Figure 9. An image taken by Pixel 10 Pro with the saturation adjusted prior to capture of the image. The C2PA manifest offers no indication that the view seen by the camera lens was edited, despite clear alterations present.

Example 4 (Figure 9) demonstrates an attack on C2PA's provenance using the Pixel 10 Pro, where image alterations and camera settings prior to image capture are not recorded in the C2PA manifest. The C2PA manifest supports recording the media's provenance, but doing so is not required by the specifications. This example illustrates a deficiency of the specifications and not an issue with Google's compliance.

In this example, the *original image* is the view seen by the camera lens. The *captured image*, a JPEG taken with a Pixel 10 Pro, is an undocumented manipulation of that original image by the trusted device. This violation of providence is different from a violation of authenticity when photographing a staged event. With this picture, we set the saturation to maximum prior to pressing the shutter button to capture the image. The photo includes a complete and valid C2PA manifest, indicating that a Pixel Camera produced the image. This pre-capture manipulation differs from ordinary changes to saturation after an image is captured, where changes are visible in the C2PA manifest, either explicitly with assertions that describe the changes made, or simply that some editing software opened and edited the picture.

Alterations using camera filters available to the user before capture, such as adjusting exposure or saturation on the capture screen, allow users intentionally or unintentionally to disguise editing. With such a drastic alteration to the image, an expert is not needed to determine that the image has been altered. However, with more subtle changes using pre-capture filters, users may be misled to believe images modified by pre-capture filters are unaltered.

Example 5: Violation of Strong File Integrity

The C2PA specifications allow for multiple exclusion ranges (Section 2), which leave parts of the file unprotected. Google’s rationale is that users may wish to anonymize GPS coordinates without violating the signature. However, this policy allows an adversary to alter the unprotected GPS coordinates without detection. In Section 13.2, we propose an alternative that allows offline redaction without violating strong file integrity.

For example, the Google Pixel 10 (software version HDR+ 1.0.821848803zd) excludes 6,741 bytes across 14 ranges. The ranges span the EXIF GPS information (131 bytes) and Cr manifest (6,610 bytes).

Example 5 (Figure 10) demonstrates this vulnerability by changing the GPS metadata of the image to a location near the Golden Gate Bridge in San Francisco. While the Adobe and CAI verifiers ignore EXIF GPS data entirely, Verifieddit and ProofMode extract and display the location based on the altered coordinates.

Because these changes occur within the excluded byte range, no online validator detects the tampering. While this behavior technically adheres to SG2 (*weak file integrity*)—because the signed bytes remain intact—it constitutes a failure of SG5 (*strong file integrity*) by permitting undetectable modifications to the file’s meaningful content. Furthermore, because validators fail to disclose that the GPS data are excluded from the Cr’s protection, users are left with a false sense of assurance regarding the image’s provenance.

Example 6: Expired Validation

Example 6 (Figure 11) illustrates the temporal fragility of C2PA validation. In January 2025, the Arizona Secretary of State released C2PA-signed media as part of a pilot

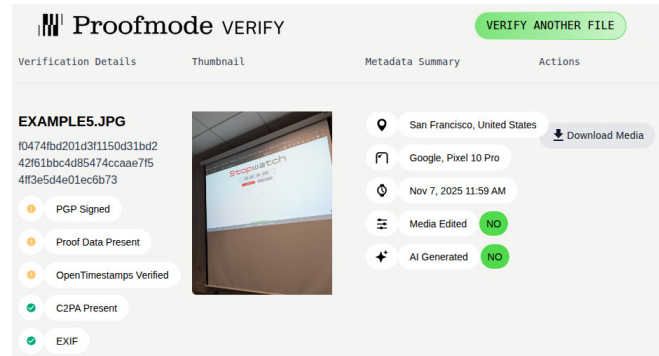


Figure 10. A photo from a Google Pixel 10. We altered the GPS coordinates to indicate a location in San Francisco. Without detecting the alteration, ProofMode’s verifier identifies the Cr as cryptographically valid and displays the false GPS coordinates.

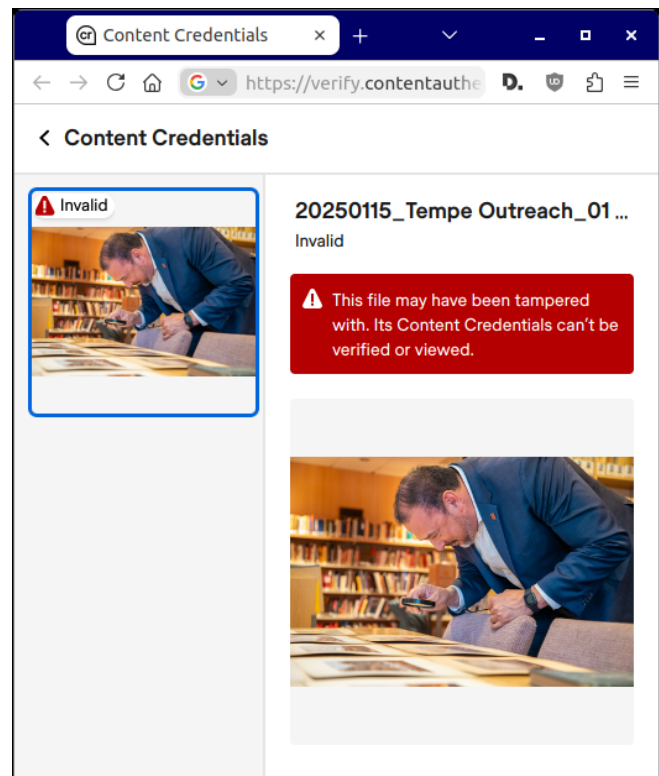


Figure 11. An image from CAI’s Content Credentials website showing the Arizona Secretary of State. The image was validated in January 2025, but fails to validate a year later, even though the file has not changed.

program [70]. At the time of this release, C2PA and CAI validators reported that the manifests were verified and valid. However, the signing certificate and the trusted timestamp certificate expired on November 18, 2025. Despite the file remaining bit-for-bit identical to its original state, validators now return conflicting and misleading results: Adobe Inspect and Verifieddit fail to locate the manifest entirely; CAI Verify warns that the file “may have been tampered with”; and ProofMode reports the manifest as present and valid.

This inconsistency constitutes a violation of SG4 (*validator consistency*), as the “truth” of the provenance data changes based on the date of inspection and the specific validator used.

This problem is not limited to Arizona’s pilot program. Other examples include the 2021 demonstrations by the *New York Times* and Starling Labs [71], [72], 2024 news reports from CBC/Radio Canada (e.g., [73]), images from OpenAI GPT-4o prior to December 2025, and some pictures signed by Google AI prior to January 2026. These Cr-signed files are no longer verifiable due to certificate expiration or incompatible version changes.

To be legally or historically useful, signed media must remain verifiable over extended durations. C2PA advocates have frequently positioned the specification as a critical tool for securing election records,³ protecting creator rights, and advising consumers with a digital “nutrition label” [74], [75]. However, there is a significant gap between regulatory requirements and current technical performance. For example, U.S. federal law requires election materials to be retained for 22 months [76] and the National Archives mandates certain records be kept “for the life of the Republic” [77]. In contrast, many C2PA implementations failed to remain verifiable for even 12 months, while most certificates in the trust list expire within 20 years (Section 9.1). Example 6 demonstrates a critical lack of long-term validation support. Without a demonstrated reliable ability to validate content years later, C2PA cannot be depended upon to verify media beyond a short duration.

Appendix G. Discussion

Dangers of Crs. When assessing an object’s provenance, users may consider C2PA Crs. Even experts find it challenging to make such judgments. The danger is that Crs may mislead users, since many users will likely accept the provenance of an object that is accompanied by a Cr. Problems from the specifications, implementations, and user interfaces result in user confusion.

For example, EXIF and C2PA timestamps serve different purposes (Section 4). An adversary can leverage inconsistencies between the EXIF and C2PA signed timestamps to amplify misinformation or disinformation related to the inconsistency. We observed similar contradictions by using older (flawed or outdated) versions of C2PA, or by signing the object using an entity that was compromised or non-compliant with C2PA’s conformance program [36].

False invalidation. An attacker could cast doubt on the integrity of a *valid* digital asset file by attaching an *invalid* manifest to the file. A similar outcome arises when the certificate of a valid signature expires. In each case, validation fails. The user will not know why the validation failed—a compromised file or an invalid manifest.

3. Securing election integrity would require much more than using a properly working provenance system.

Adoption vs. security. To enhance adoption, the C2PA specifications include many permissive and optional policies that diminish security, as do many other protocols (e.g., FIDO UAF [78]). Crucial features—such as, version numbers, and timestamps on claim signatures—are optional. Validators are not required to validate certain special manifests consistently, such as those that include deprecated assertions. When dealing with manifests that include claims with malformed fields, validators may accept the manifest, ignoring the malformed fields, or reject the entire manifest.

When evaluating a product, the conformance program checks whether the submitted specifications match C2PA’s requirements, but does not verify the implementation. Generators may use byte-range hashes, rather than more specialized hashes. Omitted by the exclusion range, the C2PA manifest store is not consistently protected by cryptographic signatures. Overly-generous exclusion ranges are permitted, allowing some content to be altered without detection. These inconsistencies can have significant implications on privacy (e.g., including in a claim the thumbnail of an asset prior to editing the asset) and on provenance (e.g., selectively excluding or including important metadata). These permissive policies create chaos for validators and reduce the value of the claims.

Appendix H. Privacy Considerations

The provenance goals of C2PA specifications inherently create tension with privacy [48]. For example, cryptographically binding an image with its metadata (e.g., camera, location, time) might reveal and endanger the photographer. A possible solution (beyond the scope of this paper) might involve introducing a system of pseudonyms throughout the C2PA ecosystem, though such a system would face significant challenges of usability and possible linkability. This appendix presents a privacy analysis of C2PA, organized around three dimensions: (1) information leakage during normal operation, (2) side-channels introduced by privacy-preserving mechanisms, and (3) disclosure through the verification process.

H.1. Revocation Checking and Privacy

For security, it is essential to check for revoked certificates. The C2PA specifications’ treatment of certificate revocation attempts to trade security for privacy. We analyze the information leakage of the three available revocation-checking mechanisms by characterizing what each protocol message reveals to potential adversaries.

OCSP (Online Certificate Status Protocol). The validator sends a request containing the serial number of the specific certificate being checked to the CA’s OCSP responder. This request might reveal to network observers and the CA: (i) that a specific certificate is being validated, (ii) the time of validation, and (iii) the validator’s network identity (IP address). Because each query identifies a single certificate,

an OCSF responder operated by or colluding with a content generator can determine exactly when and where a specific signed asset is being verified. Over time, the CA accumulates a log that maps certificates to verification events, enabling tracking of media consumption patterns.

OCSF Stapling. The generator embeds a time-stamped OCSF response in the manifest at signing time [79]. The validator checks the stapled response locally, making no outbound query. Neither the network observer nor the CA learns anything at validation time. However, OCSF stapling has three practical limitations that undermine its privacy benefit: (1) The generator must embed the status response at the time of signing or prior to distribution. If a certificate is revoked after the media has been distributed, the stale staple continues to assert validity; no mechanism exists to update distributed manifests. (2) Stapling requires the manifest to allocate padding space in the manifest for the OCSF response at signing time. We found that current generators omit this padding, rendering post-capture stapling impossible. (3) The OCSF staple is in the unprotected manifest header, allowing removal without detection and un-revoking the certificate.

Certificate Revocation Lists (CRLs). The validator downloads the complete revocation list from the CA, then checks locally whether the certificate appears on it. The CA learns only that some validator requested the list, not which certificate motivated the request. The network observer learns the same. Compared to OCSF, CRLs provide k -anonymity with respect to the checked certificate, where k equals the number of certificates issued by that CA. The privacy advantage is that a CA issuing thousands of certificates cannot distinguish which one prompted the download.

The C2PA specifications support only OCSF and OCSF stapling. C2PA v2.2 states that “the claim generator shall not use CRLs” [5, Section 14.5.2]. This prohibition removes the only mechanism that simultaneously provides meaningful revocation checking and certificate-level privacy. The result is a false dilemma in which implementers must choose between privacy (skip revocation checking entirely) and security (use OCSF, sacrificing privacy). CRLs would provide a middle path.

H.2. Linkability Through Provenance Metadata

Even when C2PA avoids direct personal identifiers, metadata embedded in manifests can enable re-identification through correlation. We analyze two linkability vectors: *cryptographic linkability* via signing certificates and *semantic linkability* via manifest contents.

If a content generator uses the same signing certificate across multiple assets, any manifest inspector can easily link all assets signed by that certificate to a common source. Google’s Pixel 10 Pro mitigates this privacy vulnerability by generating a unique leaf certificate per image (“one-and-done” certificates) [8], preventing certificate-based correlation. However, this mitigation is not required by the C2PA specifications, and generators, including software-based generators, might reuse certificates across assets.

Even with one-and-done certificates, manifests contain semi-stable fields that persist across images and can serve as quasi-identifiers. Based on our examination of Pixel 10 Pro manifests, these fields may include the claim generator name and version, device family, and action sequences (e.g., created by “Pixel Camera” followed by specific processing steps). The combination of these fields narrows the anonymity set. While any single field might be shared by millions of users, their conjunction can be highly identifying. For example, the set of users running a specific firmware build, using a particular camera application version, and producing a characteristic action sequence (e.g., “crop → enhance → export”) may be small. A platform aggregator who observes multiple images sharing this fingerprint can cluster them as originating from a single device, even without certificate linkage.

This aggregation constitutes a *mosaic effect* where individually non-identifying fields become identifying when combined with each other and with external context (e.g., posting times, community membership, or a single image that reveals a face or location). The risk is amplified in C2PA because provenance metadata is cryptographically signed and thus more durable and harder to strip than conventional EXIF metadata. We note that this problem resists easy solutions within the C2PA framework. Removing metadata improves privacy but degrades provenance, which is the core purpose of the system. This inherent tension does not reflect a design flaw, but users and policymakers should understand that C2PA-signed media carry a richer and more persistent metadata footprint than do unsigned media.

H.3. Redaction Side-Channels

The C2PA specifications support redaction of assertions from signed manifests to protect privacy (e.g., removing GPS coordinates after signing). This feature is designed to allow content anonymization without invalidating the cryptographic signature. However, the redaction mechanism introduces side-channels. When an assertion is redacted, its label remains visible in the manifest’s `redacted_assertions` field. The label identifies the type of information that was removed. For instance, redacting the `stds.schema-org.CreativeWork` label can indicate that an identity/attribution assertion existed and the creator chose to conceal their identity. Similarly, redactions to location or GPS fields might indicate that the image was taken at a sensitive or private location. In each case, the presence of a redaction is informative. The assertion that “something was removed” can increase scrutiny or suspicion, particularly in adversarial contexts. This information flow constitutes a covert channel since the redaction mechanism, designed to protect privacy, inadvertently leaks information about the creator’s privacy intentions.

C2PA manifests may also embed thumbnail assertions that capture the image at the time of claim creation or ingredient import. When a user performs privacy-motivated edits (e.g., blurring faces, cropping out identifying features, or removing license plates) and then applies a new C2PA

claim, the current image reflects the edits. However, earlier thumbnails embedded in ingredient manifests or prior claims may preserve the pre-edit content. This preservation creates a class of leakage scenarios including: 1) Pre-crop leakage. For instance, a user crops an image to remove a bystander. The ingredient manifest's thumbnail retains the original, uncropped image. 2) Pre-blur leakage. A user blurs a face for anonymity. A thumbnail generated before the edit preserves the unblurred face. 3) Ingredient chain leakage. An image passes through multiple editing stages, each generating a new claim with an ingredient reference to the prior claim. The full chain of thumbnails constitutes a visual edit history that may expose content the user intended to remove.

This risk is not unique to C2PA: any system that preserves edit history faces similar challenges. However, C2PA amplifies the risk because thumbnails are carried as part of cryptographically-verified provenance, making them more durable and more authoritative than conventional metadata. Users performing quick privacy edits on mobile devices are unlikely to realize that provenance artifacts may preserve earlier representations of their images.

H.4. Verification as Disclosure

The act of verifying a C2PA-signed asset can compromise privacy, depending on the verification method used. We analyze three verification scenarios.

Upload-to-Verify Services. Public validators operating as web applications to which users upload media for verification receive: (i) the complete media file, (ii) all provenance metadata including the full manifest and certificate chain, (iii) the user's IP address and browser fingerprint, and (iv) the timing of verification. This upload constitutes a significant privacy exposure, particularly for sensitive media (e.g., images from conflict zones or involving vulnerable individuals). Even validators that perform client-side processing using JavaScript or WebAssembly are not privacy-neutral. We observed that Adobe Inspect [22] and CAI Verify access [19] access `"/_app/version.json"` at one-minute intervals beginning when the user uploads a file. These periodic requests function as beacons, revealing to the service operator that a user is actively engaged in a verification session, and for how long.

Remote Resource Fetching. C2PA supports externally referenced assertion data ("cloud data") and remote sidecars. While the specifications state that such data are not retrieved during standard validation and are fetched only when explicitly needed [5], implementations that fetch referenced URLs enable beacon-like tracking. A content generator that in an assertion embeds a URL pointing to its own server can detect: (i) that the asset was inspected, (ii) when, (iii) from which IP address, and (iv) potentially which validator was used (via User-Agent headers).

Offline Verification. In fully offline verification, the validator checks signatures and hashes locally without making any network requests. This approach eliminates leakage to network observers and to remote services. Offline verification, however, is incompatible with OCSP and with fetching

current trust lists. This fact creates a direct conflict where the most privacy-preserving verification mode is also the least secure, because it cannot detect revoked certificates. This situation could be mitigated if the validator were to download the complete revocation list from the CA (e.g., using CRLs), then check locally whether the certificate appears on it.

H.5. Regulatory Tensions

C2PA's provenance model interacts with existing data protection regulations in ways that merit consideration. Under GDPR Article 17 and similar frameworks, data subjects may request deletion of their personal data. When C2PA manifests containing personal information (e.g., creator identity, location, device fingerprint) are distributed and embedded in media across multiple platforms, honoring erasure requests becomes practically infeasible. The cryptographic binding ensures that removing assertions invalidates the manifest, and the distributed nature of media sharing means that no single entity controls all copies. GDPR Article 5(1)(c) requires that personal data be adequate, relevant, and limited to what is necessary. The C2PA specifications do not provide guidance on which assertions constitute necessary provenance versus unnecessary personal data exposure. The decision is left entirely to generators, some of which (as we observed with the Pixel 10's initial release) might include more metadata than users might expect. Provenance metadata collected for one purpose (e.g., establishing image provenance) may be repurposed by platform aggregators for tracking, profiling, or advertising. The cryptographic durability of C2PA manifests makes such repurposing more feasible than with conventional, easily-stripped metadata. We do not claim that C2PA violates these regulations. The structural properties of C2PA's design, however, create friction with data protection principles that users, deployers, and regulators should understand.

H.6. Privacy Recommendations

We recommend:

- 1) Support CRLs for revocation checking to provide a privacy-security middle ground between OCSP (which leaks certificate identity) and skipping revocation checks entirely (which compromises all security goals).
- 2) Minimize semantic fingerprinting by recommending that generators omit or generalize claim generator version strings, firmware build numbers, and other fields that contribute to device fingerprinting without improving provenance.
- 3) Address thumbnail leakage by requiring generators to regenerate or omit thumbnails after privacy-motivated edits, and by specifying thumbnail handling guidance for privacy-sensitive workflows.
- 4) Discourage remote resource references in assertions, or require that validators not fetch external URLs without explicit user consent, to prevent web-bug-style tracking.

- 5) Specify metadata lifecycle policies that address the issue caused when cryptographic assurances expire. When a cryptographic assurance expires, the justification for retaining rich provenance metadata weakens, and platforms should be guided on what to do.
- 6) Provide privacy impact guidance to generators and deployers, addressing the interaction between C2PA's provenance model and data-protection regulations such as GDPR.

Appendix I.

History of C2PA's Claims

The C2PA specifications' goals and claims have shifted considerably. C2PA Version 1 materials presented the specifications as a broad solution for journalistic integrity, election protection, creator rights, proper attribution, and evidentiary chain-of-custody. In 2021, C2PA and CAI outreach material prominently mentioned that C2PA could help artists prevent unauthorized reuse [80], maintain attribution as media circulated online [81], and support legal workflows by preserving a reliable chain of custody [21]. Subsequent analysis, however, indicates that these ambitions were overstated [82].

In practice, social media platforms frequently remove C2PA metadata, undermining its usefulness for validating content. Manifests contained unverifiable, user-supplied claims that enable false attribution [83], [84]. Changes in the specifications rendered previously authenticated content incompatible with later validators. Moreover, because the C2PA specifications permit post-signing redactions to support privacy [5, Section 6.8], a manifest can remain valid even if portions of its history are concealed, limiting its suitability for strict forensic or legal requirements.

Beginning in 2024, the specifications moved away from these earlier ambitions. C2PA Version 2 removed much of the detailed and unverifiable metadata previously embedded in manifests, shifting the information into optional gathered assertions. User-level attribution was deliberately eliminated from the core specifications and relocated to a separate effort managed by the *Creator Assertions Working Group* (CAWG) [63].

C2PA documentation reflects this reorientation of goals. The 2021 Guiding Principles [81] and early Project Origin announcements [85] emphasized protecting the information ecosystem and supporting newsroom workflows. By 2025, the C2PA 2.2 Technical Specification [5] and NIST filings [86] focus almost entirely on identifying the origin of digital assets—e.g., what camera or generative AI software created the asset. Thus, C2PA documentation has evolved from providing provenance, attribution, and rights protection toward providing more focused aspects of provenance (avoiding claims of authenticity) and identifying AI-generated media.

Appendix J.

CPSA Source Code Snippets

We provide selected snippets from our CPSA source code. For complete details, including all of our models and shapes, see our GitHub [11].

J.1. Macro Definitions

Listing 1 defines three macros that we used to define our models. When processing CPSA source code, CPSA first expands each macro, performing a textual substitution.

```
1 ;; Definition of a claim signature
2 (defmacro (claimsignature assertions asset signer) (enc (hash assertions (hash asset)) (
  ↪ privk signer)))
3 ;; Definition of a manifest
4 (defmacro (manifest assertions asset signer) (cat (claimsignature assertions asset
  ↪ signer) assertions (hash asset)))
5 ;; Definition of a timestamp
6 (defmacro (timestamp assertions asset signer time timeauthority)
7   (cat (hash (claimsignature assertions asset signer)) time (enc (hash (hash (
  ↪ claimsignature assertions asset signer)) time) (privk timeauthority)))))
```

Listing 1. Definitions of three macros used to simplify CPSA modeling.

J.2. Protocol Models

To model the C2PA protocols in CPSA, we define roles for the Generator, TSA, and Verifier. Each role is defined by specifying what messages the role sends and receives. For example, Listings 2–5 define the five generator roles representing each of the five possible executions of the generator we used in our formal-methods analysis (Section 7). Generator0 works in protocol *C2PA-NoTimestamp*; Generator1 works in protocol *C2PA-RemoteTSA*; Generator2 works in protocol *C2PA-GeneratorSignature*; Generator3 works in protocol *C2PA-TimestampManifest*; and Generator4 works in protocol *C2PA-TimestampManifestF*.

```
1 (defrole generator0 ;; generator providing no timestamp with manifest
2 (vars (gen name) (media data) (assertions text))
3 (trace
4   (send (cat media (manifest assertions media gen)))
5   )
6 )
7
8 (defrole generator1 ;; generator providing timestamp from TSA with manifest
9 (vars (gen tsa name) (media data) (assertions time text))
10 (trace
11   (send (hash (claimsignature assertions media gen)))
12   (recv (timestamp assertions media gen time tsa))
13   (send (cat media (manifest assertions media gen) (timestamp assertions media gen time
    ↪ tsa)))
14   )
15 )
```

Listing 2. CPSA source code for the roles of Generator0 (used in *C2PA-NoTimestamp*) and Generator1 (used in *C2PA-RemoteTSA*).

```

1 ;; generator providing countersigned timestamp (our fix)
2 (defrole generator2
3 (vars (gen tsa name) (media data) (assertions time text))
4 (trace
5 (send (hash (claimsignature (cat assertions "TS") media gen)))
6 (recv (timestamp (cat assertions "TS") media gen time tsa))
7 (send (cat media (manifest (cat assertions "TS") media gen) (timestamp (cat assertions
  ↳ "TS") media gen time tsa) (enc (hash (timestamp (cat assertions "TS") media gen
  ↳ time tsa)) (privk gen))))))
8 )
9 )

```

Listing 3. CPSA source code for the role of Generator2 (used in *C2PA-GeneratorSignature*), which implements one of our two recommended improved protocols. This improved protocol includes a flag signaling when a timestamp is expected. The protocol also countersigns the timestamp when it is present.

```

1 (defrole generator3 ;; generator providing timestamp manifest
2 (vars (gen tsa name) (media data) (assertions time text))
3 (trace
4 (send (hash (claimsignature assertions media gen)))
5 (recv (timestamp assertions media gen time tsa))
6 (send (cat media (manifest assertions media gen) ;; Original Manifest
7 (manifest ;; Timestamp Manifest w/ reference to original
8 (cat (timestamp assertions media gen time tsa) (manifest assertions media gen)
  ↳ media gen))))
9 )
10 )

```

Listing 4. CPSA source code for the role of Generator3 (used in *C2PA-TimestampManifest*), which adds a timestamp to the current manifest as a signed assertion. With this flawed protocol, the timestamp manifest can be removed without detection.

```

1 (defrole generator4
2 (vars (gen tsa name) (media data) (assertions time text))
3 (trace
4 (send (hash (claimsignature (cat assertions "TS") media gen)))
5 (recv (timestamp (cat assertions "TS") media gen time tsa))
6 (send (cat media (manifest (cat assertions "TS") media gen) ;; Original Manifest
7 (manifest ;; Timestamp Manifest w/ reference to original
8 (cat (timestamp (cat assertions "TS") media gen time tsa) (manifest (cat assertions
  ↳ "TS") media gen)) media gen))))
9 )
10 )

```

Listing 5. CPSA source code for the role of Generator4 (used in *C2PA-TimestampManifestF*), which includes an assertion that a timestamp manifest should be present in the parent manifest and includes the timestamp received from the TSA in the current manifest, improving upon the protocol in Listing 4. This protocol is our main recommended improvement.

J.3. Security Goals

We define each of the security goals SG1, SG2, SG3, SG5 (Section 5) as an implication over sets of predicates. For example, Listing 6 defines security goals SG1, SG2 and SG5 for validators that do not receive a timestamp and Listing 7 defines Proposition 2 of SG3 that states that it should not be possible to remove a timestamp provided by a generator.

```
1 (defgoal c2pa
2   (forall ((media data) (assertions text) (gen name) (z strd))
3     (implies
4       (and (p "validator0" z 1) (p "validator0" "media" z media)
5         (p "validator0" "assertions" z assertions)
6         (p "validator0" "gen" z gen) (non (privk gen))))
7     (or
8       (exists ((z-0 strd))
9         (and (p "generator0" z-0 1) (p "generator0" "media" z-0 media)
10          (p "generator0" "assertions" z-0 assertions)
11          (p "generator0" "gen" z-0 gen) (prec z-0 0 z 0)))
12       (exists ((z-0 strd))
13         (and (p "generator1" z-0 1) (p "generator1" "media" z-0 media)
14          (p "generator1" "assertions" z-0 assertions)
15          (p "generator1" "gen" z-0 gen) (prec z-0 0 z 0)))
16       (exists ((z-0 strd))
17         (and (p "generator2" z-0 3) (p "generator2" "media" z-0 media)
18          (p "generator2" "assertions" z-0 assertions)
19          (p "generator2" "gen" z-0 gen) (prec z-0 2 z 0)))
20       (exists ((z-0 strd))
21         (and (p "generator3" z-0 3) (p "generator3" "media" z-0 media)
22          (p "generator3" "assertions" z-0 assertions)
23          (p "generator3" "gen" z-0 gen) (prec z-0 2 z 0)))
24       (exists ((z-0 strd))
25         (and (p "generator4" z-0 3) (p "generator4" "media" z-0 media)
26          (p "generator4" "assertions" z-0 assertions)
27          (p "generator4" "gen" z-0 gen) (prec z-0 2 z 0))))))
```

Listing 6. CPSA source code defining security goals SG1, SG2, and SG5 for validators that do not receive a timestamp stated as an implication over sets of predicates.

```
1 (defgoal c2pa_reduced
2   (forall ((media data) (assertions text) (gen name) (z strd))
3     (implies
4       (and (p "validator0" z 1) (p "validator0" "media" z media)
5         (p "validator0" "assertions" z assertions)
6         (p "validator0" "gen" z gen)
7         (non (privk gen)))
8       (false)))
9   (comment "Analysis of Proposition 2"))
```

Listing 7. CPSA source code for Proposition 2 of SG3.

Appendix K.

Example of a C2PA Validator and Manifest

The sunset image shown in the preview below is a JPEG photo that we captured using a Pixel 10 Pro on October 5, 2025. The photo includes a Cr with a single claim asserting that an authentic Pixel 10 Pro captured the image data. The CAI validator output and JSON output from the `c2patool` command line interface are shown in Figure 12.



Figure 12. CAI validator and c2patool CLI output for image of a sunset taken by a Pixel 10 Pro.

Contents

1	Introduction	1
2	Overview of the C2PA Ecosystem	3
3	Examples of Security Failures	4
3.1	Example 1: Timestamp Replacement	5
3.2	Example 2: Inconsistent Validators	5
3.3	Example 3: Revoked Certificates	6
4	C2PA Specifications	6
5	Security Goals	8
6	Adversarial Model	8
7	Formal-Methods Analysis (Summary)	9
8	Our Two Suggested Improved Protocols	9
9	Analysis of the C2PA Conformance Program	9
9.1	Certificate Authority Trust List	9
9.2	Conforming Products List	10
10	Assessing Content and Provenance: Issues, Limits, and Capabilities	10
11	Previous and Related Work	11
12	Open Problems and Future Work	11
13	Findings and Recommendations	11
13.1	Findings	11
13.2	Recommendations	12
14	Conclusion	13
	References	14
	Appendix A: Open Science	16
	Appendix B: Ethical Considerations	16
	Appendix C: Acronyms and Abbreviations	16
	Appendix D: Ladder Diagrams	17
	Appendix E: Formal-Methods Analysis	19
E.1	C2PA Strand Space	19
E.2	Formal Security Goals	20
E.3	Analysis	21
	Appendix F: Additional Examples	22
	Appendix G: Discussion	24

Appendix H: Privacy Considerations	24
H.1 Revocation Checking and Privacy	24
H.2 Linkability Through Provenance Metadata	25
H.3 Redaction Side-Channels	25
H.4 Verification as Disclosure	26
H.5 Regulatory Tensions	26
H.6 Privacy Recommendations	26
Appendix I: History of C2PA’s Claims	27
Appendix J: CPSA Source Code Snippets	28
J.1 Macro Definitions	28
J.2 Protocol Models	28
J.3 Security Goals	30
Appendix K: Example of a C2PA Validator and Manifest	31